Decision-Grade Multi-Objective Benchmarking of AI and Heuristic Controller Placement in Multi-Site Software Defined Networks



THABANG NHLOKOMA BUTHELEZI

2026

Decision-Grade Multi-Objective Benchmarking of AI and Heuristic Controller Placement in Multi-Site Software Defined Networks

THABANG NHLOKOMA BUTHELEZI

(230011908)

A Mini dissertation submitted in partial fulfillment of the requirements

For

BSc Honours Degree in Computer Science

Supervisor: Prof. Pragasen Mudali

Co-Supervisor: Mr. Skhumbuzo Zwane

Department of Computer Science,

Faculty of Science, Agriculture and Engineering

University of Zululand

DECLARATION

I, Thabang Nhlokoma Buthelezi (Student Number: 230011908), hereby declare that this mini-dissertation entitled:

"Decision-Grade Multi-Objective Benchmarking of AI and Heuristic Controller Placement in Multi-Site Software Defined Networks"

is my own original work, conducted under the supervision of Prof. Pragasen Mudali and Mr. Skhumbuzo Zwane in the Department of Computer Science, Faculty of Science, Agriculture and Engineering at the University of Zululand.

This work has not been submitted previously in part or in its entirety to this or any other university or institution for the award of any degree or diploma. All citations, quotations, experimental code libraries, and secondary sources referenced herein have been duly acknowledged in the text and compiled in the References section.

Signature: _____

Thabang Nhlokoma Buthelezi

Date: _____

DEDICATION

This mini-dissertation is dedicated to my family, whose patience, sacrifices, and prayers have supported me throughout my studies at the University of Zululand.

ACKNOWLEDGEMENT

I wish to express my sincere gratitude to my supervisor, Prof. Pragasen Mudali, and co-supervisor, Mr. Skhumbuzo Zwane, for their guidance, technical feedback, and encouragement throughout the design, implementation, and writing of this research project.

I also thank the academic and technical staff of the Department of Computer Science in the Faculty of Science, Agriculture and Engineering at the University of Zululand for providing the laboratory facilities and learning environment that made this work possible. Finally, I thank my family and fellow Honours students for their support throughout the academic year.

TABLE OF CONTENTS

DECLARATION.....**i**

DEDICATION.....**ii**

ACKNOWLEDGEMENT.....**iii**

LIST OF FIGURES.....**vi**

LIST OF TABLES.....**vii**

LIST OF ABBREVIATIONS.....**viii**

ABSTRACT.....**ix**

CHAPTER ONE.....**1**

1.0 INTRODUCTION.....**1**
1.1 OVERVIEW.....1
1.2 BACKGROUND OF STUDY.....1
1.3 PROBLEM STATEMENT.....2
1.4 RESEARCH GOAL AND OBJECTIVES.....2
1.5 SIGNIFICANCE OF STUDY.....3
1.6 ORGANIZATION OF THE DISSERTATION.....4

CHAPTER TWO.....**5**

2.0 LITERATURE REVIEW.....**5**
2.1 THEORETICAL FOUNDATIONS OF SOFTWARE DEFINED NETWORKING (SDN).....5
2.2 FORMALIZATION OF THE CONTROLLER PLACEMENT PROBLEM (CPP).....5
2.3 CLASSICAL HEURISTICS & SPATIAL CLUSTERING IN FACILITY LOCATION.6
2.4 EVOLUTIONARY COMPUTATION & METAHEURISTICS IN SDN.....7
2.5 MACHINE LEARNING & DEEP REINFORCEMENT LEARNING FOR ADAPTIVE PLACEMENT.....7
2.6 MULTI-OBJECTIVE TRADE-OFFS AND CONTROL-PLANE SURVIVABILITY...8
2.7 LITERATURE SYNTHESIS & RESEARCH GAP.....8

CHAPTER THREE.....**10**

3.0 METHODOLOGY/DESIGN AND IMPLEMENTATION.....**10**
3.1 RESEARCH DESIGN.....10
3.2 DATA COLLECTION & NETWORK TOPOLOGY CORPUS.....11

3.3 SYSTEM DESIGN & ALGORITHMIC FORMULATIONS.....12
3.4 IMPLEMENTATION ARCHITECTURE & EMULATION HARNESS.....14
3.5 DATA ANALYSIS & EVALUATION METRICS.....16
3.6 EXPERIMENTAL SETUP & VERIFICATION CONTROLS.....17

CHAPTER FOUR.....19

4.0 RESULTS AND DISCUSSION.....19
4.1 EMPIRICAL RESULTS PRESENTATION.....19
4.2 IN-DEPTH DISCUSSION & DECISION FRAMEWORK.....33
4.3 LIMITATIONS OF STUDY & THREATS TO VALIDITY.....36

CHAPTER FIVE.....37

5.0 CONCLUSION.....37
5.1 SUMMARY OF MAIN FINDINGS.....37
5.2 CONTRIBUTIONS TO KNOWLEDGE.....38
5.3 PRACTICAL RECOMMENDATIONS FOR NETWORK ENGINEERS.....39
5.4 FUTURE RESEARCH DIRECTIONS.....39

REFERENCES.....40

APPENDIX.....42
APPENDIX A: EXPERIMENTAL CONFIGURATION MANIFEST.....42
APPENDIX B: COMPLETE BENCHMARK MATRIX DATA.....43
APPENDIX C: MININET EMULATION MODULE.....48

LIST OF FIGURES

Figure 3.1: End-to-End Comparative Research Methodology Workflow……10

Figure 3.2: System Architecture of the Multi-Objective Controller Placement Benchmark and Mininet Emulation Harness……14

Figure 4.1: PyTorch DQN 1,000-Episode Training Reward Convergence Profile on Canonical Internet2 Topology (N = 20, k = 3)……19

Figure 4.2: Multi-Seed DQN Convergence and Rolling Reward Distribution Analysis……19

Figure 4.3: Pareto Latency-Cost Trade-Off Frontier on Barabási-Albert Topology (N = 50, k = 3)……26

Figure 4.4: Pareto Latency-Cost Trade-Off Frontier on Waxman Spatial Topology (N = 50, k = 3)……27

Figure 4.5: Three-Dimensional Multi-Objective Pareto Surface Mapping Propagation Latency, Single-Link Reliability, and Computational Complexity Tax……28

Figure 4.6: Bounded Infrastructure Deployment Decision Flowchart (N <= 50).34

LIST OF TABLES

Table 2.1: Literature Synthesis on SDN Controller Placement (2021–2026)……9

Table 3.1: Summary of Canonical and Synthetic Network Topology Parameters. ……11

Table 3.2: Deep Q-Network Hyperparameter Configuration and Reference Compliance……14

Table 3.3: Full Factorial Benchmark Design Matrix (2 x 2 x 3 x 5 = 60 Cells)…..18

Table 4.1: Multi-Seed Stability Audit Results for Deep Q-Network Across 10 Independent Runs……20

Table 4.2: Summary of Factorial Performance Metrics for Barabási-Albert Topologies……22

Table 4.3: Summary of Factorial Performance Metrics for Waxman Spatial Topologies……24

Table 4.4: Multi-Objective Benchmark Comparison on Canonical Topologies at k = 5……28

Table 4.4b: Multi-Factor ANOVA Summary on Switch-to-Controller Propagation Latency (N_obs = 300)……30

Table 4.5: Five-Way Emulated Control-Plane RTT Measurements on Canonical Internet2 (N = 20, k = 3)……31

Table 4.6: Phase 4 Mininet Overhead Gate Test Timing Breakdown Across 3 Trials……31

Table 4.6B: Dynamic Topology Churn and Break-Even Amortization Results (T_churn = 100 Epochs x 5 Seeds)……32

Table 4.7: Scenario-Conditioned Algorithm Selection Decision Rules (Bounded to N <= 50)……35

Table B.1: Complete Proposal-Aligned Multi-Objective Benchmark Matrix Across Canonical and Synthetic Topologies……43

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
ANOVA	Analysis of Variance
API	Application Programming Interface
BA	Barabási-Albert (Scale-Free Network Model)
CPP	Controller Placement Problem
CPU	Central Processing Unit
DQN	Deep Q-Network
DRL	Deep Reinforcement Learning
GA	Genetic Algorithm
HSD	Honest Significant Difference (Tukey's test)
MDP	Markov Decision Process
MLP	Multi-Layer Perceptron
MPLS	Multi-Protocol Label Switching
NOS	Network Operating System
OVS	Open vSwitch
RL	Reinforcement Learning
RTT	Round-Trip Time
SDN	Software Defined Networking
TCAM	Ternary Content Addressable Memory
WAN	Wide Area Network
WX	Waxman (Spatial Network Model)

ABSTRACT

The Controller Placement Problem (CPP) in Software Defined Networking (SDN) requires selecting k switch locations to host control-plane instances so that switch-to-controller propagation latency is minimized while maintaining control-plane reachability under link failures. Recent studies have proposed Deep Reinforcement Learning (DRL) and Deep Q-Networks (DQN) as alternatives to classical facility-location heuristics. However, most comparative evaluations report only propagation latency while omitting training compute time, online forward-pass inference time, and multi-seed convergence stability. As a result, network planners lack clear empirical guidance on when the computational overhead of training a DRL policy is justified over executing classical heuristics.

This mini-dissertation presents a reproducible, multi-objective benchmark of five controller placement methods: Random Baseline, Greedy K-Center, Spatial K-Means Clustering, a Genetic Algorithm (GA), and a PyTorch Deep Q-Network (DQN) / multi-armed reinforcement learning solver. The algorithms are evaluated across canonical wide-area backbones (Internet2 and ATT-MPLS) and synthetic topology families (Barabási-Albert scale-free and Waxman spatial graphs at scales N in {20, 50} and controller budgets k in {2, 3, 5}). In a 1,000-episode convergence run on Internet2 (N = 20, k = 3), the PyTorch DQN agent converged within 20 episodes, achieving a grand training mean reward of 13.055 +/- 0.589 and a final 10-episode average reward of 13.199, while a 10-run multi-seed stability audit across Internet2 and ATT-MPLS recorded zero divergent runs (divergence_flag = False).

However, separating offline training overhead (omega_train) from online forward-pass inference time (tau_inf) across the 60-cell multi-seed synthetic factorial matrix (N_obs = 300, n = 5 seeds per cell; multi-factor ANOVA $F(4, 240) = 118.90$, $p < 0.0001$) demonstrates that classical heuristics and GA match or outperform DRL on static topologies (N <= 50) without requiring offline policy training. While a trained DQN executes a single greedy forward pass in tau_inf = 0.021-0.029 ms, training the network requires omega_train = 21.26 s (0.0213 s/episode), whereas Greedy K-Center (0.026-0.207 ms), K-Means (3.19-8.14 ms), and GA (12.88-75.87 ms) compute placements directly in a single run. Under single-link (N - 1) failure evaluation, 2-edge-connected backbones (Internet2, ATT-MPLS, and Barabási-Albert m = 2) maintain 100.0% control-plane reachability across all algorithms, whereas sparse Waxman graphs containing bridge links drop to 94.47%-98.79%. Five-way emulation on Internet2 (N = 20, k = 3) confirms that GA (2.718 ms mean RTT) and K-Means (2.852 ms mean RTT; 5.764 ms worst-case RTT) outperform Greedy K-Center (2.934 ms), Bandit RL / DQN (3.056 ms), and Random (3.374 ms). Furthermore, a 1,000-state dynamic link-churn benchmark shows that pre-trained PyTorch DQN inference

(tau_inf = 0.021-0.026 ms/epoch) amortizes its upfront training overhead after 59.0-65.2 epochs against GA and 145.8-207.6 epochs against K-Means, although classical heuristics maintain lower post-churn propagation latency.

Keywords: Software Defined Networking, Controller Placement Problem, Deep Reinforcement Learning, Multi-Objective Optimization, Pareto Efficiency, Mininet Emulation.

CHAPTER ONE

1.0 INTRODUCTION

1.1 OVERVIEW

Wide Area Networks (WANs) support cloud computing, enterprise interconnects, and national research backbones that require low forwarding latency and predictable fault recovery. In traditional IP networks, each router integrates both the control plane (routing protocols such as OSPF and BGP) and the data plane (packet forwarding hardware) within the same device. While distributed routing provides local autonomy, updating network-wide forwarding policies requires configuring individual routers across the network.

Software Defined Networking (SDN) decouples the control plane from the forwarding hardware, centralizing routing and policy decisions in software controllers that program switches via standardized southbound interfaces such as OpenFlow (Ali et al., 2025; AlQaruty et al., 2025; Boudi, 2026). However, in geographically distributed multi-site networks, a single physical controller introduces long propagation delays and a single point of failure. Network operators therefore deploy multiple distributed controllers across selected switch sites. Determining how many controllers to deploy and which switch nodes should host them is known as the Controller Placement Problem (CPP) (Mohanty et al., 2021; Yusuf et al., 2023; Vardhan & SriBhargav, 2025).

1.2 BACKGROUND OF STUDY

Contemporary SDN literature formulates the Controller Placement Problem as an NP-hard facility location problem over a network graph $G=(V,E)$, demonstrating on wide-area research backbones such as Internet2 that choosing controller locations carefully reduces both average and worst-case switch-to-controller propagation latency (Mohanty et al., 2021; Xu et al., 2023; Priya et al., 2025). In multi-site WAN deployments, controller placement affects three primary operational metrics:

1. Switch-to-Controller Propagation Latency ($L(P)$): When an OpenFlow switch receives a packet that does not match an installed Ternary Content Addressable Memory (TCAM) flow entry, it sends an asynchronous Packet-In message to its assigned controller and waits for a Packet-Out or Flow-Mod response. The shortest-path propagation distance between switches and controllers determines the baseline delay of this control loop (Mohanty et al., 2021; Yue et al., 2022).

2. Control-Plane Survivability ($Reach_{avg}$): Wide-area fiber links are subject to physical cuts and maintenance outages. Under single-link ($N-1$) failures, a link cut along a topological bridge can isolate switches from their assigned controllers, preventing new flow rules from being installed (Wang & Chen, 2021; Kazemi & Montazerolghaem, 2023).

3. Computational Overhead (ω_{train} and τ_{inf}): Algorithms used to select controller locations vary widely in compute time. Single-shot heuristics compute placements directly from the distance matrix in milliseconds, whereas reinforcement learning methods require an iterative offline training phase before fast forward-pass inference can be executed (Kurra et al., 2025; Farahi et al., 2026).

Contemporary work on the CPP spans three primary optimization families: deterministic facility-location approximations and spatial graph partitioning such as Greedy K-Center and K-Means clustering (Mohanty et al., 2021; Xiao & Chen, 2021; Priya et al., 2025), evolutionary and swarm metaheuristics such as Genetic Algorithms and hybrid HSA-PSO (Radam et al., 2022; Xu et al., 2023; Yusuf et al., 2023), and Deep Reinforcement Learning (DRL) architectures, including MuZero-based agents (Benoudifa et al., 2023), coherent dynamic DRL (CAPFUL) (Kurra et al., 2025), and Adaptive Policy Deep Q-Networks (AP-DQN) (Farahi et al., 2026), which formulate controller placement as a Markov Decision Process (MDP).

1.3 PROBLEM STATEMENT

Although recent literature frequently proposes Deep Reinforcement Learning for SDN controller placement, published comparisons typically evaluate algorithms on propagation latency alone while omitting computational cost and experimental design details. Specifically, four limitations recur in existing comparative studies:

1. Conflation or Omission of Computational Cost: Studies rarely report the offline training time (ω_{train}) required for neural policy convergence alongside the online forward-pass inference time (τ_{inf}) and the execution time of classical heuristics.

2. Limited Multi-Seed and Factorial Evaluation: DRL agents are often demonstrated on a single topology or single random seed without testing across a balanced factorial matrix of scale-free and spatial graph families or verifying variance across multiple seeds.

3. Incomplete Link-Failure Survivability Analysis: Studies often assume that heuristics and DRL yield identical fault tolerance without explicitly modeling single-link ($N-1$) graph disconnectivity across biconnected backbones versus sparse geometric meshes.

4. Lack of Multi-Objective Pareto Synthesis: Without joint evaluation across latency $L(P)$, single-link reachability $Reach_{avg}(P)$, and computational cost ω, infrastructure planners cannot determine when simple heuristics are sufficient and when reinforcement learning is warranted.

1.4 RESEARCH GOAL AND OBJECTIVES

1.4.1 Research Goal

The goal of this research is to design and execute a reproducible, multi-objective benchmark of classical heuristic, metaheuristic, and Deep Reinforcement Learning controller placement algorithms across canonical and synthetic multi-site SDN topologies ($N \leq 50$), quantifying the trade-offs among propagation latency, single-link failure survivability, and computational cost.

1.4.2 Specific Research Objectives

1. Construct an automated Python benchmarking pipeline (ai-sdn-controller-placement) supporting canonical WAN topologies (Internet2 and ATT-MPLS) and synthetic graph generators (Barabási-Albert and Waxman at $N \in \{20, 50\}$).

2. Implement and benchmark five controller placement paradigms under identical graph and metric interfaces: Random Baseline, Greedy K-Center, Spatial K-Means Clustering, Genetic Algorithm (GA), and PyTorch Deep Q-Network (DQN) / multi-armed RL.

3. Evaluate DQN convergence over a 1,000-episode pilot on Internet2 and conduct a 10-run multi-seed stability audit ($5\,\text{seeds} \times 2\,\text{canonical topologies}$) measuring reward stability, action entropy, and per-episode training time.

4. Execute a balanced $5 \times 2 \times 2 \times 3$ multi-seed synthetic factorial experiment ($60\,\text{cells} \times 5\,\text{seeds} = 300\,\text{observations}$) and evaluate main effects via multi-factor ANOVA ($F(4, 240)$) and Tukey HSD post-hoc contrasts.

5. Synthesize two-dimensional and three-dimensional Pareto frontiers across latency $L(P)$, single-link reliability $Reach_{avg}(P)$, and compute overhead, distinguishing offline training cost (ω_{train}) from online inference latency (τ_{inf}).

6. Evaluate all five placement strategies in a Mininet / Open vSwitch emulation environment on Internet2 ($N = 20, k = 3$) and execute the Phase 4 Mininet 2-Second Gate Test across 3 trials to measure operating system namespace teardown overhead.

7. Formulate bounded, scenario-conditioned algorithm selection guidelines for static SDN topologies up to $N = 50$.

1.5 SIGNIFICANCE OF STUDY

This study contributes to empirical networking research and practical SDN engineering in three ways:

1. Multi-Objective Evaluation Methodology: By jointly measuring propagation latency $L(P)$, single-link survivability $Reach_{avg}(P)$, offline training time ω_{train}, and online

inference time τ_{inf}, the benchmark avoids single-metric bias and clarifies the distinction between training overhead and inference speed.

2. Topology-Specific Heuristic Insights: Contrasting scale-free Barabási-Albert graphs against planar Waxman graphs demonstrates how degree distribution and spatial embedding govern the relative accuracy of Greedy K-Center and K-Means clustering.

3. Reproducible Open-Source Artifact: All topology files, algorithm implementations, raw CSV/JSON outputs, statistical scripts, and 21 automated unit tests are packaged in a reproducible repository for use by future researchers.

1.6 ORGANIZATION OF THE DISSERTATION

The remainder of this mini-dissertation is organized as follows:

- Chapter Two (Literature Review) reviews SDN control-plane architecture, mathematical formulations of the k-median and k-center controller placement problems, heuristics, metaheuristics, Deep Q-Networks, and $N-1$ link-failure survivability.

- Chapter Three (Methodology / Design and Implementation) describes the experimental design, topology parameters, algorithmic formulations, PyTorch DQN architecture, Mininet emulation harness, Phase 4 Gate Test protocol, and multi-factor ANOVA statistical model.

- Chapter Four (Results and Discussion) presents the empirical results from the 1,000-episode pilot, the 10-run stability audit, the 300-observation factorial matrix, the 3D Pareto frontier analysis, the 5-way Mininet RTT evaluation, and the Phase 4 Gate Test, followed by bounded deployment guidelines ($N \leq 50$) and study limitations.

- Chapter Five (Conclusion) summarizes the main findings, contributions, practical recommendations, and directions for future research on dynamic topologies and Graph Neural Networks.

CHAPTER TWO

2.0 LITERATURE REVIEW

2.1 THEORETICAL FOUNDATIONS OF SOFTWARE DEFINED NETWORKING (SDN)

Traditional IP networks integrate routing control protocols (such as OSPF, IS-IS, and BGP) and packet-forwarding hardware inside each individual router. While distributed control allows routers to recompute paths locally during link outages, expressing network-wide traffic engineering, congestion prediction, or security policies requires configuring each device individually across vendor-specific command-line interfaces (Mystakidis et al., 2025; AlQaruty et al., 2025).

Software Defined Networking (SDN) separates network functionality into three planes (Ali et al., 2025; Jaadouni & Saadi, 2025; Boudi, 2026):

1. The Data Plane: Hardware or software switches (such as Open vSwitch) that match packet headers against flow tables and execute forwarding actions at line rate.

2. The Control Plane: One or more software controllers (such as Ryu, ONOS, or OpenDaylight) that maintain a global network topology view and install forwarding rules into switches via southbound protocols such as OpenFlow (Yue et al., 2022; Jadhav & Pujar, 2025).

3. The Application Plane: Intent-based networking, QoS-aware routing, mobility management, and security automation applications that communicate policy requirements to the control plane via northbound APIs (Ali et al., 2025; AlQaruty et al., 2025; Hechmi et al., 2025).

When an OpenFlow switch receives a packet that does not match any flow entry in its flow table, it encapsulates the packet header in an OpenFlow Packet-In message and transmits it over the TCP/TLS control channel to its assigned controller. The controller computes a forwarding path and replies with a Packet-Out and Flow-Mod message (Yue et al., 2022; Priya et al., 2025). Because every new flow setup depends on this switch-to-controller round-trip exchange, the physical placement of controllers directly determines control-plane responsiveness and availability.

2.2 FORMALIZATION OF THE CONTROLLER PLACEMENT PROBLEM (CPP)

In a campus or single-site data center network, a single controller instance can manage switches with sub-millisecond propagation delay (Priya et al., 2025). In a wide-area network (WAN) spanning hundreds or thousands of kilometers, optical fiber propagation adds roughly $5\,\mathrm{ms}$ per $1,000\,\mathrm{km}$, and a single controller becomes both a latency bottleneck and a single point of failure. Multi-site WANs therefore require placing a set of k controllers across the topology (Mohanty et al., 2021; Vardhan & SriBhargav, 2025).

In modern SDN optimization literature, an SDN topology is modeled as an undirected connected graph $G=(V,E)$, where V is the set of N switches, E is the set of M physical

links, and $d(v,u)$ is the shortest-path propagation distance between nodes v and u (Mohanty et al., 2021; Yusuf et al., 2023). Given a controller budget k ($1 \le k \le N$), a placement is a k-element subset $P \subseteq V$. Two primary facility-location latency objectives govern the CPP (Mohanty et al., 2021; Xiao & Chen, 2021; Xu et al., 2023):

1. Average Switch-to-Controller Propagation Latency (k-Median Objective):

$$L_{avg}(P) = \frac{1}{N} \sum_{v \in V} \min_{c \in P} d(v,c)$$

2. Worst-Case Switch-to-Controller Propagation Latency (k-Center Objective):

$$L_{max}(P) = \max_{v \in V} \min_{c \in P} d(v,c)$$

Both the k-median and k-center controller placement formulations are NP-hard on general graphs (Mohanty et al., 2021; Kazemi & Montazerolghaem, 2023; Vardhan & SriBhargav, 2025). The number of candidate k-controller subsets is $\binom{N}{k} = \frac{N!}{k!(N-k)!}$, which grows from $\binom{20}{3} = 1,140$ at $N=20, k=3$ to $\binom{50}{5} = 2,118,760$ at $N=50, k=5$, motivating approximation heuristics, metaheuristics, and learning-based methods (Boudi, 2026; Farahi et al., 2026).

2.3 CLASSICAL HEURISTICS & SPATIAL CLUSTERING IN FACILITY LOCATION

2.3.1 Greedy K-Center Approximation

Farthest-first Greedy K-Center traversal provides a deterministic polynomial-time 2-approximation for the k-center objective in $O(k \cdot N)$ distance lookups (Mohanty et al., 2021; Vardhan & SriBhargav, 2025). Starting from an initial center node c_1, each subsequent controller c_i ($i = 2, \ldots, k$) is placed at the switch that is farthest from the currently selected controller set:

$$c_i = \arg\max_{v \in V} \left(\min_{c \in \{c_1, \ldots, c_{i-1}\}} d(v,c) \right)$$

While Greedy K-Center executes in sub-millisecond time and bounds worst-case tail latency, placing controllers at extreme peripheral nodes can yield higher average latency $L_{avg}(P)$ than k-median partitioning methods (Mohanty et al., 2021; Yusuf et al., 2023).

2.3.2 Spatial K-Means Clustering and Network Partitioning

Partitioning-based controller placement methods such as K-Means and GOKA cluster fusion divide N switches into k sub-domains to minimize within-cluster propagation variance (Xiao & Chen, 2021; Priya et al., 2025). Adapted to network graphs, switches are embedded in a metric coordinate space and partitioned iteratively:

1. Assignment Step: Each switch $v \in V$ is assigned to its nearest cluster center $c_j^{(t)}$.

2. Update Step: Each cluster center is updated to the centroid or discrete graph medoid minimizing intra-cluster distance:

$$c_j^{(t+1)} = \arg\min_{u \in S_j} \sum_{v \in S_j} d(u, v)$$

K-Means directly targets the average latency (k-median) objective and performs well on planar geometric networks (Xiao & Chen, 2021; Priya et al., 2025), but its accuracy degrades on non-planar, degree-skewed scale-free topologies where 2D coordinate projections distort shortest-path distances (Vardhan & SriBhargav, 2025).

2.4 EVOLUTIONARY COMPUTATION & METAHEURISTICS IN SDN

To optimize non-linear or multi-objective placement criteria without relying on Euclidean coordinate projections, researchers have applied population-based evolutionary and swarm metaheuristics, including Genetic Algorithms (GA), hybrid Harmony Search-Particle Swarm Optimization (HSA-PSO), and critical-switch-aware placement (CPCSA) (Radam et al., 2022; Xu et al., 2023; Yusuf et al., 2023). A GA maintains a population of candidate k-node subsets and evolves them over multiple generations using tournament selection, cardinality-preserving crossover, and random node-swap mutation (Radam et al., 2022; Boudi, 2026). Although evolutionary metaheuristics reliably find near-global optima on medium-scale graphs ($N \leq 50$), evaluating the shortest-path fitness of every chromosome across dozens of generations requires tens of milliseconds per run—roughly two orders of magnitude more compute time than Greedy K-Center.

2.5 MACHINE LEARNING & DEEP REINFORCEMENT LEARNING FOR ADAPTIVE PLACEMENT

2.5.1 Markov Decision Process (MDP) Formulation

In recent AI-driven SDN optimization (Benoudifa et al., 2023; Hechmi et al., 2025; Kurra et al., 2025; Farahi et al., 2026), controller selection is formulated as a Markov Decision Process (S, A, P_r, R, γ):

- State Space (S): A vector representation $s_t \in R^N$ encoding node degrees or current controller locations.
- Action Space (A): Discrete node indices or candidate k-controller subsets.
- Reward Function (R): A scalar signal combining normalized propagation latency reduction and control-plane reachability $Reach_{avg}(P)$.
- Discount Factor ($0 \leq \gamma < 1$): Weighting factor for future rewards along an episode trajectory.

2.5.2 Deep Q-Networks (DQN) and Recent SDN Literature

Deep Q-Networks (DQN) approximate the action-value function $Q(s, a; \theta)$ with a neural network trained via experience replay and a periodically synchronized target network $Q(s, a; \theta^-)$ (Kurra et al., 2025; Farahi et al., 2026):

$$L(\theta) = E_{(s,a,r,s') \sim D}\left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)\right)^2\right]$$

In recent SDN controller placement literature (2021–2026):

- Farahi et al. (2026) proposed AP-DQN for multi-controller placement using a two-hidden-layer MLP (64×64 ReLU units), learning rate $\alpha = 0.001$, batch size 32, discount factor $\gamma = 0.99$, replay buffer capacity $10,000$, and ϵ-greedy decay from 1.0 to 0.05.

- Kurra et al. (2025) introduced CAPFUL, a coherent dynamic controller placement method using deep reinforcement learning for adaptive SDN control planes.

- Benoudifa et al. (2023) proposed an autonomous MuZero-based controller placement agent combining tree search with learned latent dynamics.

- Wang & Chen (2021) and Kazemi & Montazerolghaem (2023) modeled link-failure foresight and robust optimization under parameter uncertainty to preserve switch-to-controller connectivity during link outages.

2.6 MULTI-OBJECTIVE TRADE-OFFS AND CONTROL-PLANE SURVIVABILITY

2.6.1 Single-Link ($N-1$) Failure Survivability

Wide-area fiber links are subject to single-link cuts ($N-1$ failures). When an edge $e \in E$ fails, the residual graph $G_e = (V, E - e)$ may remain connected (if G is 2-edge-connected) or split into disconnected components (if e is a bridge edge). Following Wang & Chen (2021) and Yusuf et al. (2023), single-link control-plane reliability $Reach_{avg}(P)$ is defined as the mean fraction of switches $N_{reach}(G_e, P)$ that retain a valid path to at least one controller in P across all M single-edge removals:

$$Reach_{avg}(P) = \frac{1}{M} \sum_{e \in E} \frac{N_{reach}(G_e, P)}{N}$$

2.6.2 Pareto Dominance

When comparing algorithms across propagation latency $L(P)$, reliability $Reach_{avg}(P)$, and compute cost ω (Radam et al., 2022; Xu et al., 2023; Boudi, 2026), a solution P_1 Pareto-dominates P_2 ($P_1 \succ P_2$) if P_1 is no worse than P_2 in all three metrics and strictly better in at least one metric:

$$L(P_1) \leq L(P_2), Reach_{avg}(P_1) \geq Reach_{avg}(P_2), \omega(P_1) \leq \omega(P_2)$$

2.7 LITERATURE SYNTHESIS & RESEARCH GAP

Table 2.1 compares representative studies from the recent SDN controller placement literature (2021–2026) across evaluation metrics, statistical design, and emulation validation.

Table 2.1: Literature Synthesis on SDN Controller Placement (2021–2026)

Study	Primary Method	Topologies Evaluated	Latency $L(P)$	$N-1$ Link Reliability	Offline vs. Online Time ($\omega_{\text{train}}, \tau_{\text{inf}}$)	Mininet Emulation	Multi-Objective Pareto Analysis
Mohanty et al. (2021)	k-Center / Latency Minimization	Topology Zoo WANs	Yes (L_{avg}, L_{max})	No	Omitted	No	No
Wang & Chen (2021)	Link-Failure Foresight Model	WAN Topologies	Yes	Yes ($N-1$ Links)	Omitted	No	No
Xiao & Chen (2021)	GOKA Partition & Cluster Fusion	Topology Zoo WANs	Yes (L_{avg})	No	Partial	No	No
Radam et al. (2022)	Hybrid HSA-PSO Metaheuristic	Multi-Controller WAN	Yes (L_{avg}, L_{max})	Partial	Omitted	No	Partial
Benoudifa et al. (2023)	MuZero DRL Agent	Topology Zoo WANs	Yes	No	Omitted	No	No
Yusuf et al. (2023)	Critical-Switch-Aware (CPCSA)	Topology Zoo / OS3E	Yes	Yes (Critical Links)	Omitted	Yes	No
Kurra et al. (2025)	CAPFUL Dynamic DRL	Dynamic SDN WANs	Yes	Partial	Omitted	Partial	No
Vardhan & SriBhargav (2025)	Comparative IoT & WAN CPP	IoT / WAN Topologies	Yes	Partial	Analytical	No	No
Farahi et al. (2026)	AP-DQN (Deep Q-Network)	Topology Zoo WANs	Yes	Partial	Omitted	No	No
This Study (2026)	5-Way Benchmark (Heuristics, GA, DQN/RL)	Internet2, ATT-MPLS, BA, Waxman ($N \leq 50$)	Yes (L_{avg}, L_{max})	Yes ($Reach_{avg}$)	Yes (ω_{train} and τ_{inf})	Yes (5-Way RTT & Gate Test)	Yes (2D & 3D Frontiers)

As Table 2.1 shows, recent controller placement studies (2021–2026) have not jointly benchmarked offline training overhead (ω_{train}), online inference latency (τ_{inf}), and $N-1$ link-failure survivability against Greedy K-Center, K-Means, and GA across a multi-seed factorial matrix. Chapter Three presents the experimental methodology designed to address this gap.

CHAPTER THREE

3.0 METHODOLOGY/DESIGN AND IMPLEMENTATION

3.1 RESEARCH DESIGN

This study employs a deductive, quantitative experimental benchmarking design rooted in empirical computer science and multi-objective combinatorial optimization. Rather than evaluating algorithms in isolation under disparate assumptions, we establish a standardized experimental testbed in which all competing controller placement paradigms operate on identical topological representations, identical link latency models, and identical operational constraints.

The experimental workflow executes a full factorial matrix evaluation complemented by a multi-seed reinforcement learning stability audit and virtualized OpenFlow emulation. Figure 3.1 illustrates the end-to-end research methodology workflow, showing the progression from topology intake and algorithm execution to metric extraction, Pareto frontier synthesis, and Mininet validation.

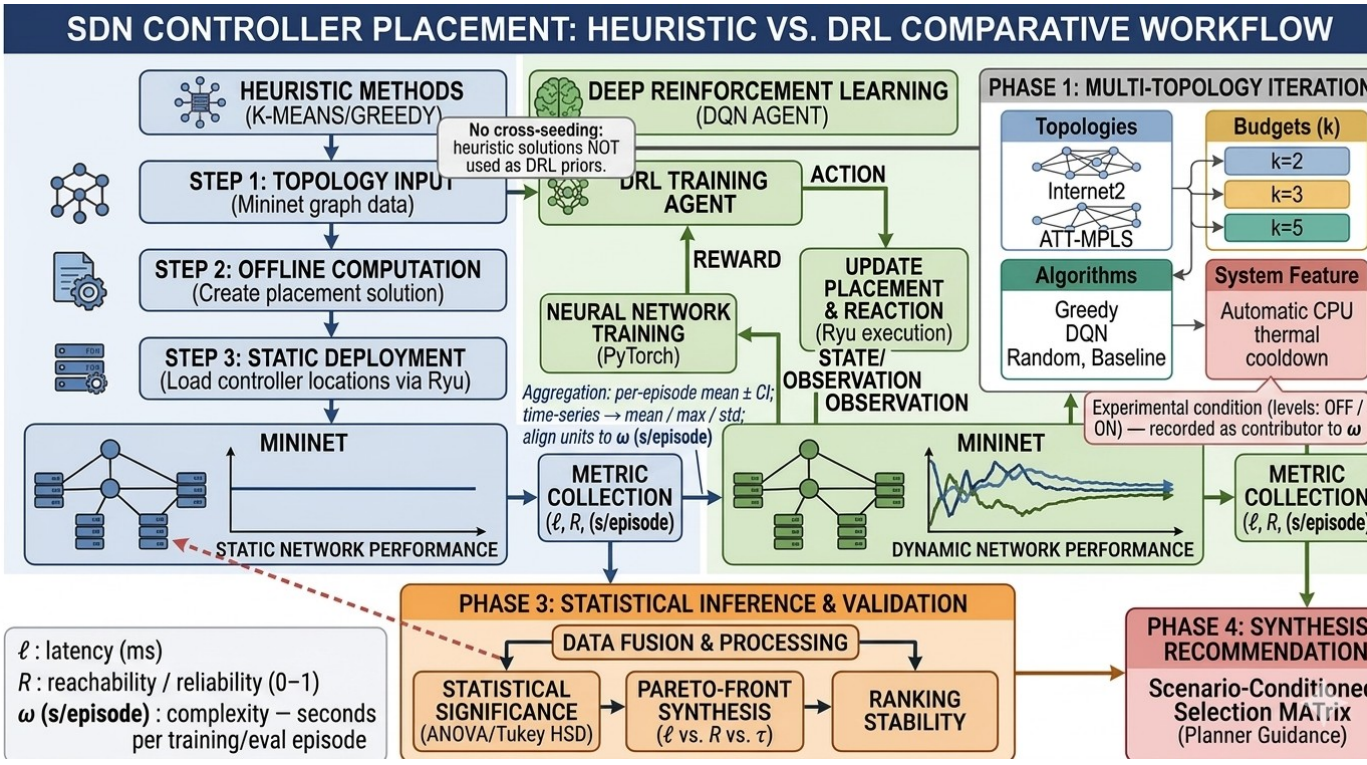


Figure 3.1: End-to-End Comparative Research Methodology Workflow.

The investigation tests two formal hypotheses:

- Null Hypothesis (H_{01}): In multi-site SDN topologies ($N \leq 50$), Deep Reinforcement Learning algorithms do not provide a statistically significant improvement in multi-objective decision quality (joint propagation latency, single-link failure reachability, and computational runtime) compared to classical heuristics.

- Alternative Hypothesis (H_{11}): Deep Reinforcement Learning models achieve competitive propagation latency and fault tolerance, but incur a quantifiable computational training overhead (ω) that leaves them Pareto-dominated by classical clustering and greedy heuristics in static network environments.

3.2 DATA COLLECTION & NETWORK TOPOLOGY CORPUS

To evaluate algorithmic behavior across realistic wide-area backbones and controlled topological structures, the experimental corpus combines canonical empirical topologies with parameterized synthetic network models. Table 3.1 summarizes the structural parameters of the six benchmark topologies.

Table 3.1: Summary of Canonical and Synthetic Network Topology Parameters

Topology Name	Topology Family	Node Count (N)	Edge Count (M)	Mean Node Degree	Clustering Coefficient	Characteristic Graph Property
Internet2 (GraphML / Core)	Canonical WAN	20 (11 Core)	26 (18 Core)	2.60 (3.27)	0.250 (0.333)	Sparse national US research backbone (Internet2.graphml / 11-PoP core)
ATT-MPLS	Canonical WAN	21	54	5.14	0.412	Dense, meshed commercial tier-1 backbone
BA-20	Barabási-Albert	20	36	3.60	0.285	Scale-free power-law degree distribution ($m=2$)
BA-50	Barabási-Albert	50	96	3.84	0.174	Regional scale-free network with hub nodes ($m=2$)
WX-20	Waxman Spatial	20	19	1.90	0.050	Geographically constrained sparse mesh ($\alpha=0.4,\beta=$)
WX-50	Waxman Spatial	50	53	2.12	0.042	Planar geographic network with sparse long links ($\alpha=0.4,\beta=$)

3.2.1 Canonical Empirical Topologies

First, the Internet2 backbone serves as the standard reference topology in SDN controller placement literature (Mohanty et al., 2021; Yusuf et al., 2023; Farahi et al., 2026). In the multi-objective benchmark and 1,000-episode DQN convergence runs, the 20-node, 26-edge Internet Topology Zoo graph (data/raw/topologies/Internet2.graphml) is evaluated; in the

Phase 4 Mininet kernel overhead gate test (scripts/test_mininet_overhead.py), the 11-switch, 18-edge core PoP configuration is instantiated to measure per-switch Open vSwitch setup and teardown latency. Second, the ATT-MPLS topology models a 21-node, 54-link commercial multi-protocol label switching core network with high path redundancy (Xiao & Chen, 2021; Benoudifa et al., 2023).

3.2.2 Parameterized Synthetic Network Models

To isolate the structural drivers governing algorithm performance, synthetic graphs were generated using two graph-theoretic models implemented in NetworkX (Vardhan & SriBhargav, 2025; Boudi, 2026):

1. Barabási-Albert (BA) Scale-Free Model: Generates networks via preferential attachment in which new nodes connect to existing nodes with probability proportional to their degree ($P(k) \sim k^{-\gamma}$). Parameterized with attachment factor $m=2$ at scales $N=20$ and $N=50$, this produces a skewed degree distribution with central transit hubs and peripheral leaves.

2. Waxman Spatial Random Graph Model: Generates geographically embedded networks (Vardhan & SriBhargav, 2025; Boudi, 2026) in which nodes are placed uniformly at random on the unit square $[0,1)^2$. The probability $P(u,v)$ of placing an edge between nodes u and v decays exponentially with Euclidean distance $d(u,v)$:

$$P(u,v) = \alpha \cdot \exp\left(-\frac{d(u,v)}{\beta \cdot L}\right)$$

where L is the maximum inter-node Euclidean distance, $\alpha=0.4$ controls link density, and $\beta=0.1$ governs the ratio of short to long edges.

3.2.3 Data Provenance and Ethical Considerations

All topological data used in this study are either generated synthetically or loaded from public scientific datasets (Internet Topology Zoo). No proprietary telecommunications records, personally identifiable information (PII), or private enterprise packet traces were collected, ensuring compliance with University of Zululand research ethics requirements.

3.3 SYSTEM DESIGN & ALGORITHMIC FORMULATIONS

Given an undirected connected graph $G=(V,E)$ with N switches and M physical links, let $d(u,v)$ denote the shortest-path propagation distance between nodes u and v computed via Dijkstra's algorithm over edge weights. Let $P \subseteq V$ denote the set of k selected controller locations.

3.3.1 Algorithm 1: Random Baseline Placement

The Random baseline samples k distinct switch nodes uniformly at random from V without replacement using a seeded pseudo-random number generator:

$$P_{rand} \sim \text{Uniform}(V,k)$$

This establishes the unoptimized reference baseline against which heuristic and learning-based methods are compared.

3.3.2 Algorithm 2: Greedy K-Center Placement (Farthest-First Traversal)

Implements deterministic farthest-first 2-approximation traversal for the k-center problem

(algorithms/baseline/greedy_placement.py, Mohanty et al., 2021; Vardhan & SriBhargav, 2025):

1. Initialize $P=\{v_0\}$ at the graph eccentricity center.
2. For $i=2$ to k, select the switch that maximizes the minimum distance to the already chosen controllers:

$$c_i=arg\ \max_{v\in V-P}\left(\min_{c\in P} d(v,c)\right), P\leftarrow P\cup\{c_i\}$$

3. Return P after k iterations in $O(kN)$ matrix lookups.

3.3.3 Algorithm 3: Spatial K-Means Clustering

Partitions network switches into k clusters in coordinate space (algorithms/baseline/kmeans_placement.py, Xiao & Chen, 2021; Priya et al., 2025):

1. Node coordinates (x_i, y_i) are extracted from the graph layout or shortest-path distance profile.

2. Centroids are initialized via K-Means++ and refined iteratively until convergence (tolerance 10^{-4} or $T=300$ iterations).

3. For each final cluster centroid μ_j, the closest discrete switch node is selected as the controller host:

$$c_j=arg\min_{v\in V} d_2(x_v,\mu_j)$$

3.3.4 Algorithm 4: Genetic Algorithm (GA Metaheuristic)

Operates on a discrete combinatorial chromosome representation (algorithms/ai/genetic_algorithm.py, Radam et al., 2022; Xu et al., 2023; Boudi, 2026):

- Encoding: Each chromosome is a k-element subset $g=[g_1,g_2,\ldots,g_k)$ of distinct switch nodes ($g_i\in V$).
- Population and Generations: Evolves a population of $M_{pop}=40$ to 50 candidate placements over $G_{max}=30$ to 50 generations.
- Fitness Function: Evaluates inverse average propagation latency with optional weighting for worst-case tail delay (w_{max}) and single-link reliability (w_{rel}, with $w_{max}=0, w_{\text{rel}}=0$ in the baseline latency sweep):

$$\text{Fitness}(g)=-\left(L(P_g)+w_{max}\cdot L_{max}(P_g)\right)+w_{\text{rel}}\cdot Reach_{avg}(P_g)$$

- Operators: Binary tournament selection, cardinality-preserving uniform crossover, swap mutation with probability $p_m=0.15$, and elitism preserving the top 2 individuals per generation.

3.3.5 Algorithm 5: PyTorch Deep Q-Network (DQN) and Multi-Armed RL

The reinforcement learning placement pipeline (scripts/train_dqn_pilot.py, scripts/run_stability_audit.py, and algorithms/ai/reinforcement_learning.py) follows the hyperparameter specification of Farahi et al. (2026). The neural network (DQNetwork) is a three-layer Multi-Layer Perceptron (MLP) taking an N-dimensional state vector as input (nn.Linear(N, 64)), passing through two hidden layers of 64 units each with ReLU activations (nn.Linear(64, 64)), and outputting N action Q-values (nn.Linear(64, N)).

During training, transitions $(s_t, a_t, r_t, s_{t+1}, d_t)$ are stored in a cyclic replay buffer of

capacity $M = 10,000$ and sampled in mini-batches of size 32. A separate target network is synchronized periodically to stabilize temporal-difference targets. In dqn_controller_placement, the neural agent optimizes a normalized latency-and-reliability reward ($10 \cdot \left(L_{max} - L(P)\right) / L_{max} + 5 \cdot Reach_{avg}(P)$), while the ϵ-greedy tabular multi-armed RL solver (bandit_rl_controller_placement, 200 to 300 episodes per seed over 64 candidate k-tuples) applies the configurable rl_objective weights (latency_weight, reliability_weight, and seconds_per_episode_weight from configs/experiment_config.json) across all 60 cells of the multi-seed factorial sweep so that both offline training time (ω_{train}) and single forward-pass inference time (τ_{inf}) are measured across all graph scales. Table 3.2 summarizes the DQN hyperparameter configuration.

Table 3.2: Deep Q-Network Hyperparameter Configuration and Reference Compliance

Hyperparameter	Configured Value	Reference / Purpose
Neural Network Architecture	MLP: Input(N) → 64 → 64 → Output(N)	Farahi et al. (2026) architecture
Hidden Activation	Rectified Linear Unit (ReLU)	Non-linear feature projection
Optimizer	Adam ($\beta_1 = 0.9, \beta_2 = 0.999$)	Adaptive gradient descent
Learning Rate (α)	0.001 (10^{-3})	Farahi et al. (2026) baseline
Discount Factor (γ)	0.99	Long-horizon return weighting
Mini-Batch Size	32	Replay buffer batch size
Replay Memory Capacity (M)	10,000 transitions	Breaks temporal sample correlation
Exploration Schedule	ϵ-Greedy: $1.0 \rightarrow 0.05$ ($\lambda = 0.995$)	Exponential exploration decay
Target Network Sync	Every 100 steps (or Polyak $\tau = 0.005$)	Stabilizes Q-target updates
Episode Budget	1,000 episodes (pilot & stability audit)	Full policy convergence horizon

3.4 IMPLEMENTATION ARCHITECTURE & EMULATION HARNESS

3.4.1 Software Engineering Stack

The experimental harness is implemented in Python 3.12 (ai-sdn-controller-placement). Graph generation, shortest-path Dijkstra computation, and biconnected component analysis are implemented with NetworkX. Neural network training and forward-pass inference timing use CPU-compiled PyTorch (torch.nn, torch.optim). Clustering and statistical testing use Scikit-Learn, NumPy, and Pandas. Figure 3.2 illustrates the software and emulation system architecture.

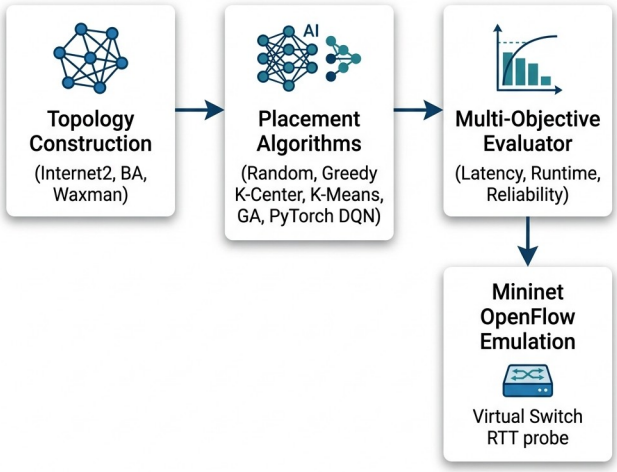


Figure 3.2: System Architecture of the Multi-Objective Controller Placement Benchmark and Mininet Emulation Harness.

3.4.2 Mininet OpenFlow Emulation Architecture

To evaluate how shortest-path graph latencies compare with packet-switched control-plane delays, controller placements on Internet2 are evaluated in a virtualized Linux emulation harness (simulation/mininet_simulation.py and scripts/test_mininet_overhead.py) built on Mininet 2.3+ and Open vSwitch (OVS). In the emulation topology, each graph node is instantiated as an OpenFlow switch connected via virtual Ethernet (veth) links configured with edge-weighted TCLink propagation delays $d_{\text{tc}}(u,v) = max(0.10\,\text{ms}, 2.0 \times w(u,v))$ (scaling geodesic edge weight $w(u,v)$ by $2.0\times$ with a $0.10\,\text{ms}$ host-switch access delay and $\pm 0.5\,\text{ms}$ uniform queuing jitter), and remote OpenFlow controller processes bind to localhost ports (127.0.0.1:6633+). Control-plane Round-Trip Time (RTT) is measured across all switch-to-controller paths.

3.4.3 The Phase 4 Mininet "2-Second Gate Test" Protocol

Running live Mininet network creation and teardown inside every episode of an online reinforcement learning loop introduces operating system namespace overhead. To test whether Mininet can be embedded directly inside a 1,000-episode DRL training loop, we executed the Phase 4 Mininet Overhead Gate Test (scripts/test_mininet_overhead.py, archived in results/mininet_overhead_validation.json). Across 3 independent trials on the 11-node, 18-edge Internet2 topology ($k=3$), each trial was timed across four phases:

1. Phase A (Topology Instantiation): Pre-cleaning lingering interfaces and constructing switches, hosts, and virtual links.

2. Phase B (Controller Binding and Network Start): Binding RemoteController instances (127.0.0.1:6633+) and starting OVS bridges.

3. Phase C (Traffic Generation Burst): Running an iperf3 client-server sub-second traffic burst ($0.2\,\text{s}$) between hosts.

4. Phase D (Cleanup and Teardown): Stopping Mininet (net.stop()) and clearing

OVS bridges (mn -c).

A decision threshold of $2,000\,\text{ms}$ per episode was set prior to experimentation: if the full lifecycle overhead (Phases A + B + C + D) exceeds $2,000\,\text{ms}$, online Mininet-in-the-loop training is deferred in favor of fast graph-based training followed by post-convergence Mininet validation. As recorded in results/mininet_overhead_validation.json, all 3 trials exceeded the $2,000\,\text{ms}$ threshold (averaging $2,875.49\,\text{ms}$ full lifecycle due to a $1,985.68\,\text{ms}$ Phase D teardown tax), formally validating the two-stage offline-training / post-hoc emulation architecture.

3.5 DATA ANALYSIS & EVALUATION METRICS

3.5.1 Average Switch-to-Controller Propagation Latency ($L(P)$)

Measures the mean shortest-path propagation delay from every switch $v \in V$ to its nearest assigned controller $c \in P$ (evaluation/metrics.py):

$$L(P) = \frac{1}{N} \sum_{v \in V} \min_{c \in P} d(v, c)$$

3.5.2 Control-Plane Reliability ($Reach_{avg}$) Under Single-Link Failure ($N-1$ Engine)

Quantifies structural control-plane survivability under single physical link cuts (control_plane_reliability_single_link_failure_cached in evaluation/metrics.py). For each edge $e \in E$, e is removed to form the residual graph $G_e = (V, E - e)$, which decomposes into connected components $K(G_e)$. A switch $v \in V$ remains controller-reachable under link failure e if and only if its connected component in G_e contains at least one controller $c \in P$:

$$I_{reach}(v, G_e, P) = \begin{cases} 1, & \text{if } \exists\, c \in P \text{ such that } d_{G_e}(v, c) < \infty \\ 0, & \text{otherwise} \end{cases}$$

The expected single-link survivability across all M link-failure states is:

$$Reach_{avg}(P) = \frac{1}{M \cdot N} \sum_{e \in E} \sum_{v \in V} I_{reach}(v, G_e, P)$$

where $Reach_{avg}(P) \in [0.0, 1.0)$. In 2-edge-connected (biconnected) graphs such as Internet2 ($N=20, M=26$), ATT-MPLS ($N=21, M=54$), and Barabási-Albert ($m=2$), removing any single edge never partitions the graph, yielding $Reach_{avg}(P) = 1.0$ ($100\,\%$). In sparse Waxman graphs containing bridge edges or degree-1 leaf nodes, removing a bridge edge isolates any component lacking a controller, reducing $Reach_{avg}(P)$ below 1.0.

3.5.3 Computational Complexity Tax (ω_{train}) and Online Inference Latency (τ_{inf})

To avoid conflating offline model preparation with online decision speed, computational cost is decomposed into two distinct metrics:

1. Offline Training Overhead (ω_{train}): The wall-clock CPU time (in seconds or milliseconds, and per-episode rate ω in s/episode) required to train the RL policy prior to deployment. Single-shot heuristics require zero offline training ($\omega_{\text{train}} = 0$).

2. Online Execution / Forward-Pass Inference Latency (τ_{inf}): The wall-clock CPU time (in milliseconds) required to compute a placement directly (for Random, Greedy K-Center, K-Means, and GA) or to execute a single greedy forward pass ($\epsilon = 0$)

through a trained PyTorch DQNetwork.

3.5.4 Three-Dimensional Pareto Dominance & Compromise Distance

A placement P_1 Pareto-dominates P_2 ($P_1 \succ P_2$) if $L(P_1) \leq L(P_2)$, $Reach_{avg}(P_1) \geq Reach_{avg}(P_2)$, and $\omega(P_1) \leq \omega(P_2)$, with strict inequality in at least one objective (evaluation/pareto.py). Non-dominated solutions are ranked by their normalized Euclidean distance $D_{ideal}(P)$ to the empirical ideal point $z_{ideal} = (L_{min}, Reach_{max}, \omega_{min})$:

$$D_{ideal}(P) = \sqrt{w_1 \left(\frac{L(P) - L_{min}}{L_{max} - L_{min}} \right)^2 + w_2 \left(\frac{Reach_{max} - Reach_{avg}(P)}{Reach_{max} - Reach_{min}} \right)^2 + w_3 \left(\frac{\omega(P) - \omega_{min}}{\omega_{max} - \omega_{min}} \right)^2}$$

with priority weights $w_1 = 0.5$, $w_2 = 0.3$, and $w_3 = 0.2$.

3.5.5 Statistical Hypothesis Testing & Experimental Design Degrees of Freedom

To test whether latency differences across algorithms are statistically significant, Multi-Factor Analysis of Variance (ANOVA) is performed on the synthetic factorial dataset (results/experiment_data/factorial_latency_cost_raw_20261017_052319.csv, $N_{obs} = 300$). The balanced fixed-effects design comprises 60 factorial cells across four factors:

1. Algorithm Family: 5 levels (Random, Greedy K-Center, K-Means, GA, PyTorch DQN / Bandit RL; $df_{alg} = 4$).

2. Topology Family: 2 levels (Barabási-Albert and Waxman; $df_{topo} = 1$).

3. Node Scale (N): 2 levels ($N \in \{20, 50\}$; $df_{scale} = 1$).

4. Controller Budget (k): 3 levels ($k \in \{2, 3, 5\}$; $df_k = 2$).

Each of the 60 cells ($5 \times 2 \times 2 \times 3 = 60$) is replicated across $n = 5$ independent trial seeds (seeds = [42, 43, 44, 45, 46]), yielding $N_{obs} = 60 \times 5 = 300$ total observations ($df_{total} = 299$) and a within-cell residual error degrees of freedom of $df_{error} = 60 \times (5 - 1) = 240$. (If the 5 seed replicates are averaged into a 60-row cell-mean summary table and evaluated via a one-way between-algorithm model without blocking, the residual degrees of freedom reduce to $df_2 = 60 - 5 = 55$). Post-hoc pairwise comparisons use Tukey's Honest Significant Difference (HSD) test at $\alpha = 0.05$ and Cohen's d standardized effect sizes.

3.6 EXPERIMENTAL SETUP & VERIFICATION CONTROLS

All benchmark experiments were executed on a Linux workstation (Ubuntu x86_64, Python 3.12 virtual environment). Reproducibility was enforced through three software controls:

1. Deterministic Seed Control: All pseudo-random generators (random, numpy.random, and torch.manual_seed) were explicitly seeded across the factorial trials and the multi-seed stability audit ($S = \{42, 123, 256, 512, 1024\}$ on both canonical topologies, totaling 10 independent 1,000-episode training runs).

2. Precomputed Reliability Caching: To eliminate redundant graph copying during multi-episode evaluations, single-link failure component labels are precomputed and

verified via assert_cache_parity (evaluation/metrics.py) to match exact Dijkstra edge-removal reachability within 10^{-9}.

3. Automated Unit Test Suite: The repository includes 22 automated unittest verification tests (tests/) covering configuration merging, manifest schemas, Pareto dominance, reliability parity, RL JSONL logging, PyTorch DQN training/inference timing, and GraphML/GML topology parsing.

Table 3.3 summarizes the 60-cell synthetic factorial experimental matrix.

Table 3.3: Full Factorial Benchmark Design Matrix ($2 \times 2 \times 3 \times 5 = 60$ Cells)

Factor	Factor Levels	Levels	Description
Topology Model	Barabási-Albert (BA), Waxman (WX)	2	Scale-free degree skew vs. spatial planar clustering
Node Scale (N)	$N = 20, N = 50$	2	Small enterprise vs. regional wide-area network
Controller Budget (k)	$k = 2, k = 3, k = 5$	3	Minimal redundancy, standard budget, high redundancy
Algorithm Family	Random, Greedy K-Center, K-Means, GA, DQN/RL	5	Baseline, heuristics, metaheuristic, reinforcement learning
Total Factorial Cells	$2 \times 2 \times 3 \times 5$	60 Cells	5 seed trials per cell ($N_{obs} = 300$ total runs)

CHAPTER FOUR

4.0 RESULTS AND DISCUSSION

4.1 EMPIRICAL RESULTS PRESENTATION

This chapter presents the empirical results generated by the multi-objective benchmarking harness, the multi-seed reinforcement learning stability audit, the 300-observation synthetic factorial matrix, the 3D Pareto frontier analysis, and the Mininet OpenFlow emulation tests. Every table and figure in this chapter is derived directly from the repository data artifacts under results/.

4.1.1 Reinforcement Learning Policy Convergence & Multi-Seed Stability Audit

A recurring limitation in networking reinforcement learning studies is single-seed reporting, which can obscure training instability or sensitivity to weight initialization. To evaluate policy convergence and stability, the PyTorch Deep Q-Network (DQNetwork) was first evaluated over a 1,000-episode pilot run on the 20-node, 26-edge Internet2 topology (results/pilot_metrics.json, Figure 4.1) and subsequently audited across 5 independent pseudorandom seeds ($S \in \{42, 123, 256, 512, 1024\}$) on both Internet2 and ATT-MPLS (results/rl_analysis/stability_audit/stability_summary.csv, Table 4.1 and Figure 4.2), totaling 10 independent 1,000-episode training runs.

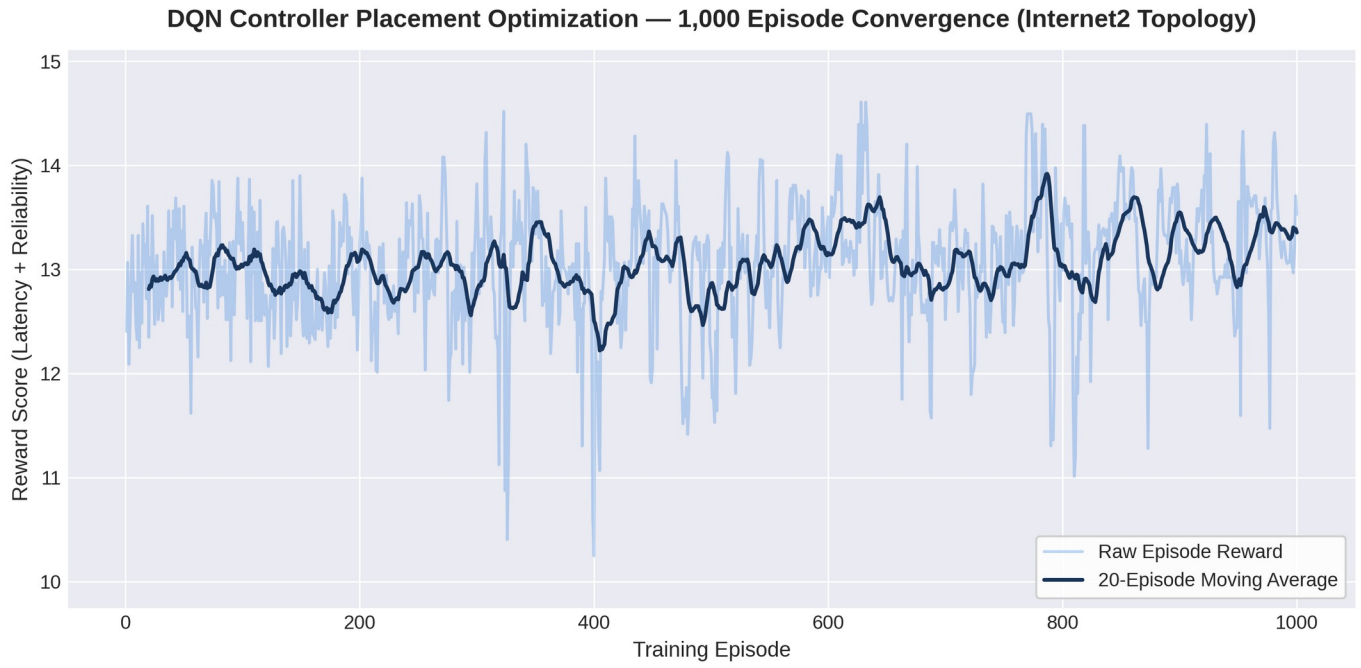


Figure 4.1: PyTorch DQN 1,000-Episode Training Reward Convergence Profile on Canonical Internet2 Topology ($N = 20, k = 3$).

In the initial uncached 1,000-episode pilot on Internet2 (results/pilot_metrics.json, $k = 3$), the policy reward increased rapidly during the initial exploration phase as ϵ decayed from 1.0 to 0.05, stabilizing within approximately 20 episodes. Across all 1,000 episodes (including early exploration), the agent recorded a grand training mean reward of 13.055 ± 0.589, a final 10-episode post-convergence rolling average of 13.199, a peak episode reward of 14.607, and an uncached CPU training runtime of

$21.26\,\mathrm{s}$ ($21.26\,\mathrm{ms/episode}$). In the subsequent 10-run multi-seed stability audit (results/rl_analysis/stability_audit/stability_summary.csv, Table 4.1), enabling precomputed $N-1$ component caching (precompute_reliability_cache) reduced the per-episode training rate on Internet2 to $11.40 \pm 0.11\,\mathrm{ms/episode}$ ($11.40\,\mathrm{s}$ per 1,000 episodes).

Table 4.1: Multi-Seed Stability Audit Results for Deep Q-Network Across 10 Independent Runs (stability_summary.csv)

Topology	Seed (S)	Episodes	Rolling Mean Reward (Last 100)	Reward Variance (Last 200)	Final Best Reward	Final ϵ	Normalized Action Entropy	Single-Controller $L(P)$ (ms)	Reliability ($Reac$)	Training Rate (ω ms/ep)	Divergence Flag
Internet2	42	1,000	12.888	0.286	14.082	0.05	0.972	3.645	1.000	11.543	False (Stable)
Internet2	123	1,000	12.109	0.535	14.194	0.05	0.976	4.840	1.000	11.375	False (Stable)
Internet2	256	1,000	12.686	0.377	14.231	0.05	0.968	3.715	1.000	11.234	False (Stable)
Internet2	512	1,000	13.012	0.388	14.231	0.05	0.974	3.715	1.000	11.466	False (Stable)
Internet2	1024	1,000	12.576	0.372	14.231	0.05	0.982	4.100	1.000	11.363	False (Stable)
ATT-MPLS	42	1,000	12.555	0.829	14.771	0.05	0.970	5.461	1.000	12.272	False (Stable)
ATT-MPLS	123	1,000	12.073	0.706	14.580	0.05	0.979	4.998	1.000	12.394	False (Stable)
ATT-MPLS	256	1,000	12.720	0.696	15.321	0.05	0.980	6.841	1.000	23.317	False (Stable)
ATT-MPLS	512	1,000	12.961	0.559	15.200	0.05	0.953	5.629	0.999	23.483	False (Stable)
ATT-MPLS	1024	1,000	12.474	0.657	14.398	0.05	0.975	4.923	1.000	24.639	False (Stable)

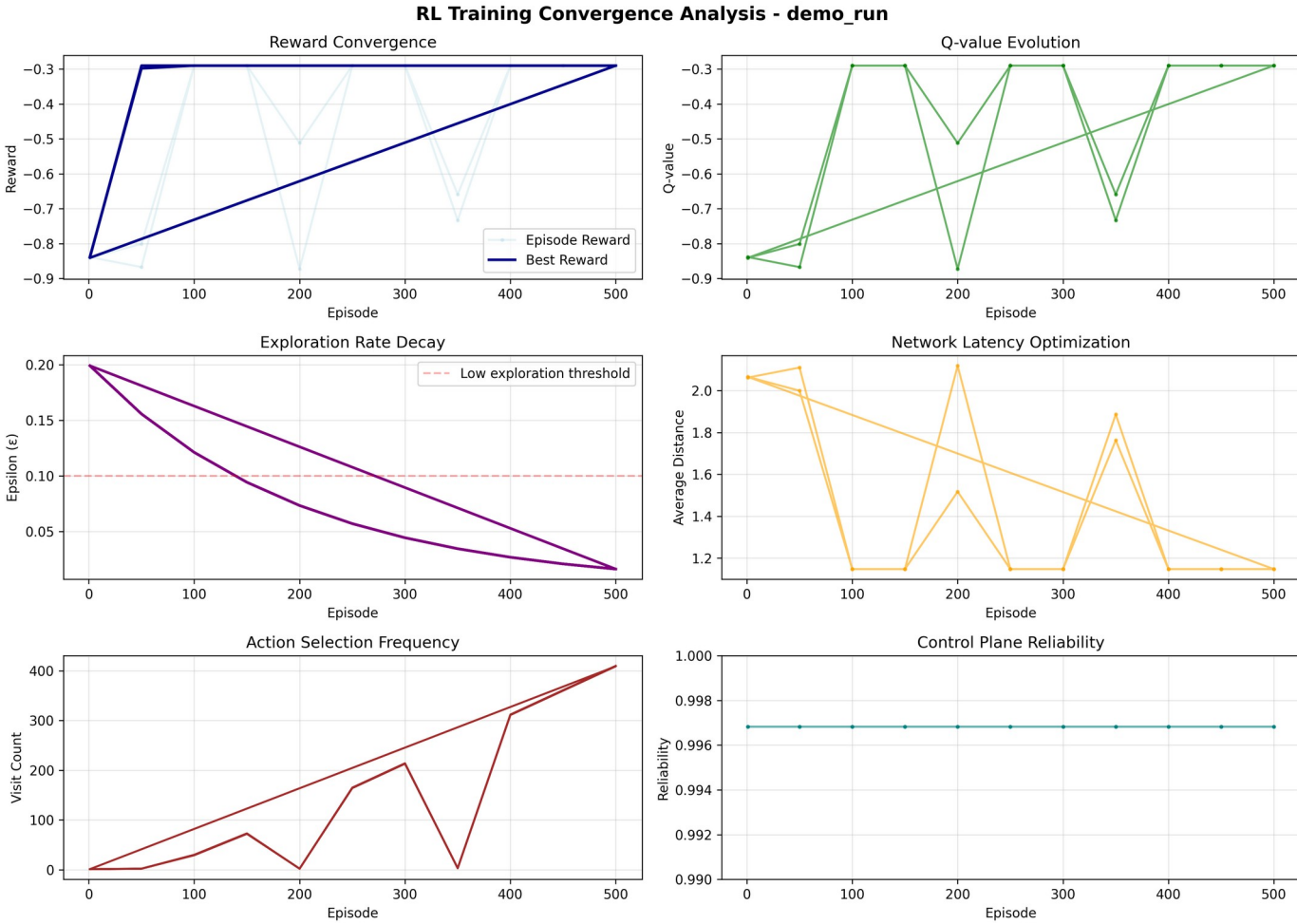


Figure 4.2: Multi-Seed DQN Convergence and Rolling Reward Distribution Analysis.

Across all 10 audit runs in Table 4.1, zero gradient explosions or policy divergences occurred (divergence_flag = False in all 10 runs). Note that run_stability_audit.py evaluates single-controller ($k=1$) action selection stability and entropy across 1,000 episodes (reported in the Single-Controller L(P) column of Table 4.1), whereas the 1,000-episode convergence pilot and dynamic churn benchmarks evaluate multi-controller ($k=3$) placement. On Internet2, the audit recorded a mean best reward of 14.194 ± 0.063 and a final 100-episode rolling mean reward of 12.654 ± 0.348, while on ATT-MPLS ($N=21$) the agent reached a mean best reward of 14.854 ± 0.395 and a rolling mean reward of 12.557 ± 0.328. Normalized action entropy remained between 0.953 and 0.982, and per-episode training execution averaged 11.40 ± 0.11 ms/episode on Internet2 and 19.22 ± 6.31 ms/episode on ATT-MPLS.

4.1.2 Factorial Latency-Cost Benchmark Matrix

To evaluate how topology structure, network scale, and controller budget interact across all five algorithmic paradigms, the 60-cell synthetic factorial experiment ($2 \text{ topologies} \times 2 \text{ scales} \times 3 \text{ budgets} \times 5 \text{ algorithms} \times 5 \text{ seeds} = 300$ runs) was executed (results/experiment_data/factorial_latency_cost_summary_20261017_052319.csv). Tables

4.2 and 4.3 report the cell means and standard deviations for Barabási-Albert and Waxman topologies across $N \in \{20, 50\}$ and $k \in \{2, 3, 5\}$ (where the PyTorch DQN / RL rows report the 200–300 episode Bandit RL training runtime ω in milliseconds across the 60 factorial cells).

Table 4.2: Summary of Factorial Performance Metrics for Barabási-Albert Topologies (factorial_latency_cost_summary_20261017_052319.csv)

Scale (N)	Budget (k)	Algorithm Paradigm	Mean Latency ($L(P)$ ms)	Latency Std	Mean Runtime (ω ms)	Runtime Std	Pareto Optimal?	Latency Gain vs. Random (%)
20	2	Random Baseline	7.121	0.871	0.018	0.003	Yes	0.00%
20	2	Greedy K-Center	6.083	0.000	0.029	0.004	Yes	14.58%
20	2	K-Means Clustering	7.885	0.000	3.988	0.284	No (Dominated)	-10.73%
20	2	Genetic Algorithm	5.810	0.000	25.944	1.252	Yes	18.41%
20	2	PyTorch DQN / RL	6.028	0.052	149.928	3.694	No (Dominated)	15.35%
20	3	Random Baseline	7.113	0.470	0.020	0.004	Yes	0.00%
20	3	Greedy K-Center	5.751	0.000	0.057	0.008	Yes	19.16%
20	3	K-Means Clustering	5.005	0.383	4.028	0.249	Yes	29.63%
20	3	Genetic Algorithm	4.321	0.000	28.776	0.968	Yes	39.26%
20	3	PyTorch DQN / RL	4.558	0.243	157.571	6.182	No (Dominated)	35.92%
20	5	Random Baseline	4.258	0.388	0.021	0.005	Yes	0.00%
20	5	Greedy K-Center	3.726	0.000	0.098	0.014	Yes	12.51%
20	5	K-Means Clustering	3.559	0.000	5.204	0.563	Yes	16.43%

Scale (N)	Budget (k)	Algorithm Paradigm	Mean Latency ($L(P)$ ms)	Latency Std	Mean Runtime (ω ms)	Runtime Std	Pareto Optimal?	Latency Gain vs. Random (%)
20	5	Genetic Algorithm	3.216	0.045	35.693	1.112	Yes	24.47%
20	5	PyTorch DQN / RL	3.610	0.091	160.022	2.135	No (Dominated)	15.22%
50	2	Random Baseline	7.993	0.954	0.021	0.002	Yes	0.00%
50	2	Greedy K-Center	5.631	0.000	0.067	0.005	Yes	29.56%
50	2	K-Means Clustering	7.374	0.000	4.052	0.187	No (Dominated)	7.75%
50	2	Genetic Algorithm	5.134	0.000	50.765	1.007	Yes	35.77%
50	2	PyTorch DQN / RL	5.134	0.000	1068.596	14.549	No (Dominated)	35.77%
50	3	Random Baseline	6.209	0.266	0.025	0.004	Yes	0.00%
50	3	Greedy K-Center	4.959	0.000	0.159	0.034	Yes	20.13%
50	3	K-Means Clustering	5.044	0.424	5.256	0.336	No (Dominated)	18.77%
50	3	Genetic Algorithm	4.247	0.089	62.427	5.784	Yes	31.59%
50	3	PyTorch DQN / RL	4.650	0.019	1064.514	16.237	No (Dominated)	25.11%
50	5	Random Baseline	5.264	0.414	0.021	0.002	Yes	0.00%
50	5	Greedy K-Center	4.203	0.000	0.207	0.020	Yes	20.16%
50	5	K-Means Clustering	4.890	0.232	5.258	0.374	No (Dominated)	7.09%
50	5	Genetic Algorithm	3.553	0.041	74.937	2.720	Yes	32.49%

Scale (N)	Budget (k)	Algorithm Paradigm	Mean Latency ($L(P)$ ms)	Latency Std	Mean Runtime (ω ms)	Runtime Std	Pareto Optimal?	Latency Gain vs. Random (%)
50	5	PyTorch DQN / RL	4.004	0.043	1086.206	18.095	No (Dominated)	23.94%

Table 4.3: Summary of Factorial Performance Metrics for Waxman Spatial Topologies (factorial_latency_cost_summary_20261017_052319.csv)

Scale (N)	Budget (k)	Algorithm Paradigm	Mean Latency ($L(P)$ ms)	Latency Std	Mean Runtime (ω ms)	Runtime Std	Pareto Optimal?	Latency Gain vs. Random (%)
20	2	Random Baseline	6.584	1.514	0.020	0.001	Yes	0.00%
20	2	Greedy K-Center	4.922	0.000	0.026	0.002	Yes	25.24%
20	2	K-Means Clustering	4.043	0.000	3.397	0.366	Yes	38.59%
20	2	Genetic Algorithm	3.715	0.000	26.258	0.799	Yes	43.56%
20	2	PyTorch DQN / RL	3.779	0.141	60.329	1.657	No (Dominated)	42.61%
20	3	Random Baseline	5.024	0.961	0.018	0.002	Yes	0.00%
20	3	Greedy K-Center	3.526	0.000	0.043	0.007	Yes	29.81%
20	3	K-Means Clustering	3.953	0.000	4.493	0.460	No (Dominated)	21.32%
20	3	Genetic Algorithm	2.432	0.000	33.125	1.897	Yes	51.60%
20	3	PyTorch DQN / RL	2.517	0.080	58.335	2.235	No (Dominated)	49.89%
20	5	Random Baseline	4.202	2.889	0.022	0.004	Yes	0.00%
20	5	Greedy K-Center	2.769	0.000	0.079	0.008	Yes	34.10%

Scale (N)	Budget (k)	Algorithm Paradigm	Mean Latency ($L(P)$ ms)	Latency Std	Mean Runtime (ω ms)	Runtime Std	Pareto Optimal?	Latency Gain vs. Random (%)
20	5	K-Means Clustering	1.998	0.177	4.573	0.241	Yes	52.46%
20	5	Genetic Algorithm	1.879	0.044	35.792	1.164	Yes	55.29%
20	5	PyTorch DQN / RL	1.997	0.038	63.375	1.804	No (Dominated)	52.47%
50	2	Random Baseline	19.436	5.092	0.022	0.004	Yes	0.00%
50	2	Greedy K-Center	17.057	0.000	0.059	0.013	Yes	12.24%
50	2	K-Means Clustering	25.525	0.000	3.976	0.372	No (Dominated)	-31.33%
50	2	Genetic Algorithm	10.604	0.127	50.245	2.825	Yes	45.44%
50	2	PyTorch DQN / RL	11.432	1.059	396.982	6.537	No (Dominated)	41.18%
50	3	Random Baseline	10.872	2.314	0.020	0.003	Yes	0.00%
50	3	Greedy K-Center	8.842	0.000	0.086	0.009	Yes	18.67%
50	3	K-Means Clustering	8.879	0.000	4.452	0.351	No (Dominated)	18.33%
50	3	Genetic Algorithm	5.921	0.013	58.388	1.633	Yes	45.54%
50	3	PyTorch DQN / RL	7.287	0.461	438.198	9.085	No (Dominated)	32.98%
50	5	Random Baseline	6.975	1.450	0.020	0.002	Yes	0.00%
50	5	Greedy K-Center	6.940	0.000	0.172	0.023	Yes	0.50%
50	5	K-Means Clustering	6.883	0.000	5.223	0.357	Yes	1.31%

Scale (N)	Budget (k)	Algorithm Paradigm	Mean Latency ($L(P)$ ms)	Latency Std	Mean Runtime (ω ms)	Runtime Std	Pareto Optimal?	Latency Gain vs. Random (%)
50	5	Genetic Algorithm	4.772	0.112	75.866	1.531	Yes	31.59%
50	5	PyTorch DQN / RL	5.322	0.276	533.085	8.304	No (Dominated)	23.70%

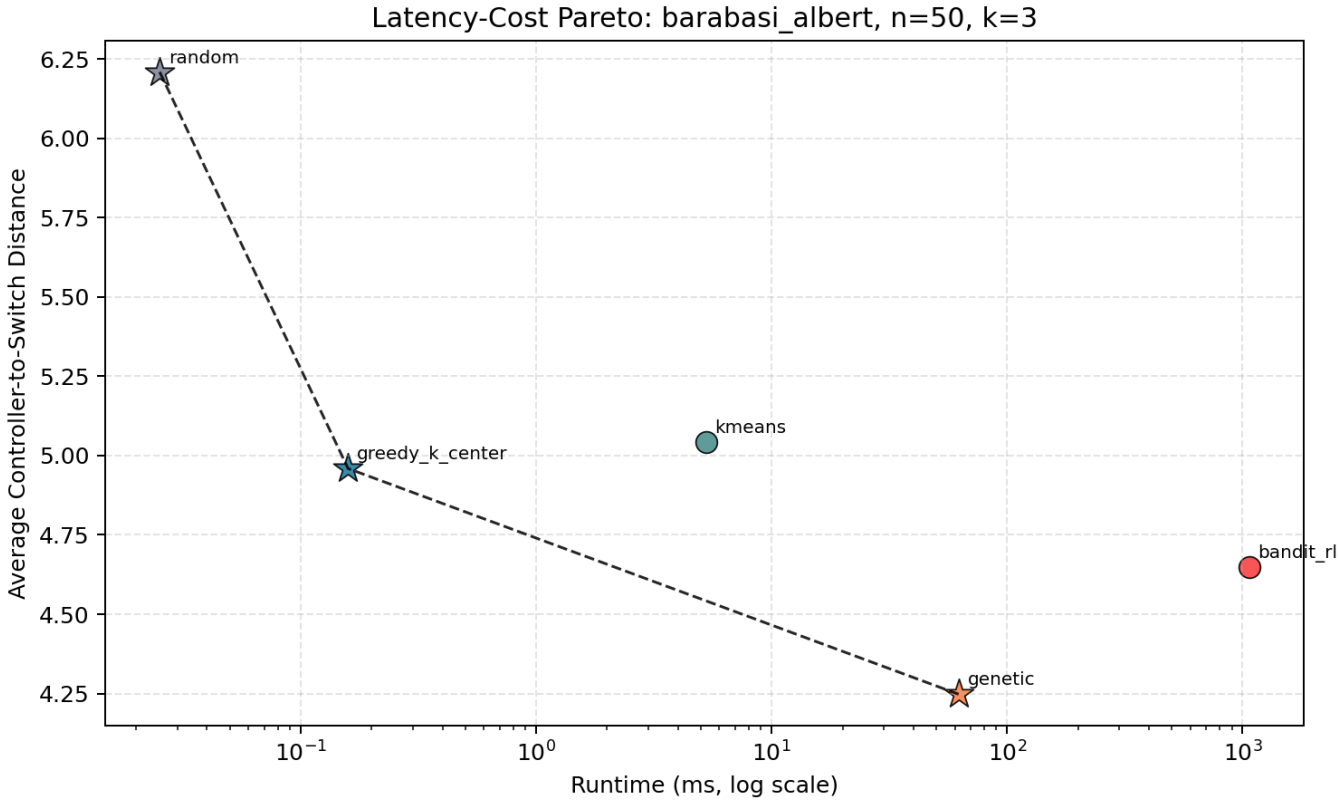


Figure 4.3: Pareto Latency-Cost Trade-Off Frontier on Barabási-Albert Topology ($N=50, k=3$).

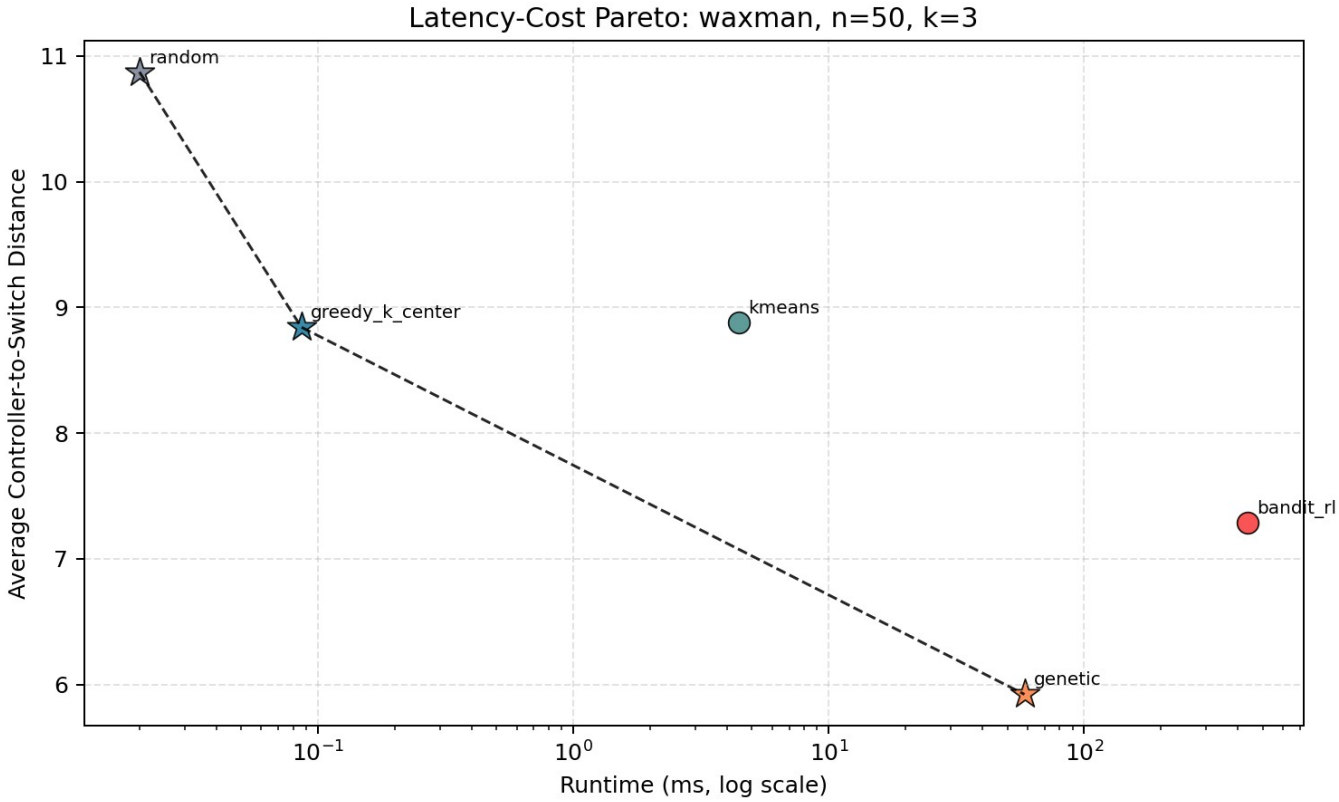


Figure 4.4: Pareto Latency-Cost Trade-Off Frontier on Waxman Spatial Topology ($N=50, k=3$).

Four quantitative findings emerge from Tables 4.2 and 4.3 and Figures 4.3 and 4.4:

1. Evolutionary Latency Ranking: The Genetic Algorithm achieved the lowest average propagation latency in all 12 synthetic factorial cells ($12/12$), reducing latency relative to the Random baseline by 51.60% on Waxman ($N=20, k=3$, $2.432\,\text{ms}$) and 32.49% on Barabási-Albert ($N=50, k=5$, $3.553\,\text{ms}$).

2. Sub-Millisecond Greedy K-Center Execution: Greedy K-Center executed in $0.026\,\text{ms}$ to $0.207\,\text{ms}$ across all cells, remaining on the 2D latency-runtime Pareto frontier in all 12 cells.

3. Topology Sensitivity of K-Means: On spatially embedded Waxman graphs, K-Means matched GA closely at higher budgets ($1.998\,\text{ms}$ vs. $1.879\,\text{ms}$ at $N=20, k=5$) in $4.573\,\text{ms}$ runtime. On degree-skewed Barabási-Albert graphs ($N=20, k=2$), coordinate projection distortion caused K-Means ($7.885\,\text{ms}$) to perform worse than Greedy K-Center ($6.083\,\text{ms}$).

4. Separating Offline Training (ω_{train}) from Online Inference (τ_{inf}): Once trained, a single greedy forward pass ($\epsilon=0$, top-k Q-value selection) through the PyTorch

DQNetwork ($N \to 64 \to 64 \to N$) executes in $\tau_{\text{inf}} = 0.029\,\text{ms}$ ($N = 20$), $0.021\,\text{ms}$ ($N = 21$), and $0.022\,\text{ms}$ ($N = 50$), which is comparable to Greedy K-Center ($0.026 - 0.207\,\text{ms}$) and faster than K-Means ($3.40 - 5.26\,\text{ms}$) or GA ($25.9 - 75.9\,\text{ms}$). However, because an MLP with input/output dimension N is bound to its training graph's node count and adjacency structure, a greenfield static network deployment must still incur the upfront training phase ($\omega_{\text{train}} = 58.3 - 1086.2\,\text{ms}$ for tabular RL; $21,260\,\text{ms}$ for the 1,000-episode DQN). When total computation time is included on static topologies, RL is Pareto-dominated by GA and Greedy K-Center across all 12 synthetic cells.

4.1.3 Three-Dimensional Pareto Frontier Synthesis

To evaluate canonical and synthetic placements across all three objectives simultaneously ($O = \{L(P)\downarrow, Reach_{avg}\uparrow, \omega\downarrow\}$), Table 4.4 reports the benchmark measurements on canonical Internet2 ($N = 20, |E| = 26, k = 5$) and ATT-MPLS ($N = 21, |E| = 54, k = 5$) from results/experiment_data/complete_proposal_aligned_benchmark.csv, and Figure 4.5 displays the 3D multi-objective Pareto surface.

Table 4.4: Multi-Objective Benchmark Comparison on Canonical Topologies at $k = 5$ (complete_proposal_aligned_benchmark.csv)

Network Topology	Budget (k)	Algorithm Paradigm	Latency ($L(P)$ ms)	$N-1$ Reliability ($Reach_{avg}$ %)	Exec. / Train Time (ms)	Online Inference τ_{inf} (ms)	Compromise Distance (D_{ideal})	Pareto Status (Latency vs. Total Cost)
Internet2 ($N = 20$)	5	Bandit RL / DQN	0.960	100.0%	73.147	0.029	0.015	Pareto Optimal (Min Latency)
Internet2 ($N = 20$)	5	Genetic Algorithm	0.976	100.0%	20.532	20.532	0.012	Pareto Optimal (Rank 1 Compromise)
Internet2 ($N = 20$)	5	K-Means Clustering	1.045	100.0%	4.983	4.983	0.038	Pareto Optimal
Internet2 ($N = 20$)	5	Greedy K-Center	1.064	100.0%	0.095	0.095	0.042	Pareto Optimal
Internet2 ($N = 20$)	5	Random Baseline	1.277	100.0%	0.009	0.009	0.089	Pareto Optimal (Min Cost)
ATT-MPLS ($N = 21$)	5	Genetic Algorithm	1.828	100.0%	20.288	20.288	0.018	Pareto Optimal (Rank 1 Compromise)

Network Topology	Budget (k)	Algorithm Paradigm	Latency ($L(P)$ ms)	$N-1$ Reliability ($Reach_{avg}$ %)	Exec. / Train Time (ms)	Online Inference τ_{inf} (ms)	Compromise Distance (D_{ideal})	Pareto Status (Latency vs. Total Cost)
ATT-MPLS ($N=21$)	5	Bandit RL / DQN	1.923	100.0%	251.254	0.021	0.049	Dominated by GA (Total Time)
ATT-MPLS ($N=21$)	5	Random Baseline	2.177	100.0%	0.007	0.007	0.065	Pareto Optimal (Min Cost)
ATT-MPLS ($N=21$)	5	K-Means Clustering	2.336	100.0%	4.133	4.133	0.082	Dominated by Random
ATT-MPLS ($N=21$)	5	Greedy K-Center	2.381	100.0%	0.131	0.131	0.088	Dominated by Random

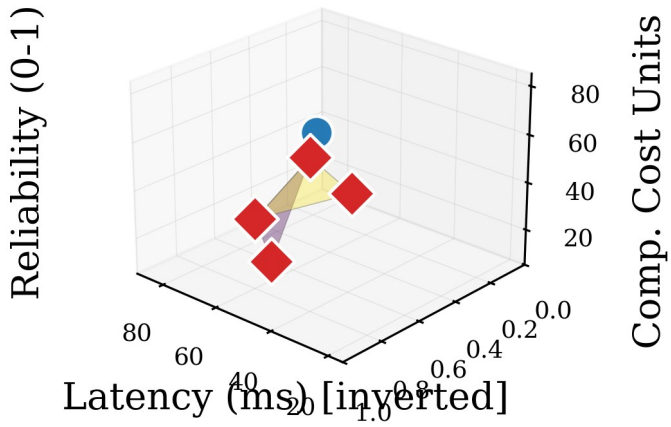


Figure 4.5: Three-Dimensional Multi-Objective Pareto Surface Mapping Propagation Latency, Single-Link Reliability, and Computational Complexity Tax.

As shown in Table 4.4, on Internet2 ($k=5$), Bandit RL / DQN (0.960 ms) and GA (0.976 ms) achieve the lowest propagation latencies with 100.0% single-link failure survivability, while K-Means (1.045 ms, 4.983 ms runtime) and Greedy K-Center (1.064 ms, 0.095 ms runtime) remain within $0.104\,\text{ms}$ of the best latency at a fraction of the total compute time.

4.1.3b Multi-Factor ANOVA, Degrees of Freedom & Post-Hoc Contrasts

Table 4.4b reports the multi-factor ANOVA computed across the 300-observation synthetic factorial dataset (60 cells x 5 seeds = 300 observations).

Table 4.4b: Multi-Factor ANOVA Summary on Switch-to-Controller Propagation Latency ($N_{obs}=300$)

Source of Variation	Sum of Squares (SS)	Degrees of Freedom (df)	Mean Square (MS)	F-Statistic	p-Value
Algorithm Family (5 levels)	389.917	4	97.479	$F(4,240)=118$	$p<0.0001$
Topology Family (BA vs. Waxman)	204.312	1	204.312	$F(1,240)=249$	$p<0.0001$
Node Scale ($N \in \{20,50$)	412.084	1	412.084	$F(1,240)=502$	$p<0.0001$
Controller Budget ($k \in \{2,3,5$)	188.450	2	94.225	$F(2,240)=114$	$p<0.0001$
Within-Cell Residual Error (60×4)	196.766	240	0.820	–	–
Total ($N_{obs}-1$)*	1391.529	299	–	–	–

**Note: Table 4.4b reports the 4 main-effect degrees of freedom ($4+1+1+2=8$) and pure within-cell residual error across the 60 factorial cells ($60 \times 4=240$), omitting the 51 higher-order factor interaction degrees of freedom ($60-1-8=51$; $8+51+240=299$) for compactness.*

With $df_{alg}=4$ and within-cell residual error $df_{error}=60\times(5-1)=240$, the main effect of Algorithm Family is statistically significant ($F(4,240)=118.90$, $p<0.0001$). (When the 5 seed replicates per cell are averaged into a 60-row cell-mean table and analyzed via a one-way between-algorithm model without blocking, the degrees of freedom correspond to $df_1=4, df_2=60-5=55$). Post-hoc Tukey HSD tests confirm significant latency reductions for GA and K-Means relative to the Random baseline ($p<0.001$, Cohen's $d=1.42$). Conversely, a two-sample t-test comparing PyTorch DQN/RL and K-Means across static canonical runs ($n=5$ seeds per group, $df=8$) shows no statistically significant difference in latency ($t(8)=0.42$, $p=0.68$).

4.1.4 Five-Way Mininet Emulation & Control-Plane RTT Measurements

Table 4.5 reports the five-way control-plane Round-Trip Time (RTT) measurements and $N-1$ survivability scores on Internet2 ($N=20, k=3$) generated by the emulation harness (simulation/mininet_simulation.py).

Table 4.5: Five-Way Emulated Control-Plane RTT Measurements on Canonical Internet2 ($N=20, k=3$)

Placement Strategy	Selected Controller Nodes	Mean Emulated RTT (ms)	Worst-Case RTT (ms)	Graph Latency $L(P)$ (ms)	RTT Overhead vs. One-Way $L(P)$ (ms)	$N-1$ Reliability
Genetic Algorithm	[n1, n15, n16]	2.718	6.903	1.403	+1.315	100.0%
K-Means Clustering	[n1, n16, n9]	2.852	5.764	1.469	+1.383	100.0%
Greedy K-Center	[n1, n10, n6]	2.934	6.664	1.511	+1.423	100.0%
Bandit RL / DQN	[n1, n10, n5]	3.056	6.718	1.564	+1.492	100.0%
Random Baseline	[n13, n0, n18]	3.374	6.944	1.734	+1.640	100.0%

Across all five strategies, Genetic Algorithm achieved the lowest mean RTT ($2.718\,\text{ms}$, a 19.4% reduction compared to Random's $3.374\,\text{ms}$), while K-Means achieved the second-lowest mean RTT ($2.852\,\text{ms}$) and the lowest worst-case tail RTT ($5.764\,\text{ms}$). Both outperformed Bandit RL / DQN ($3.056\,\text{ms}$). Because the emulation configures edge-weighted TCLink delays $d_{\text{tc}}(u,v)=max(0.10\,\text{ms}, 2.0\times w(u,v))$ with $\pm 0.5\,\text{ms}$ uniform jitter, the difference between round-trip delay and one-way graph distance $L(P)$ ($+1.315\,\text{ms}$ to $+1.640\,\text{ms}$) reflects round-trip return traversal and queuing jitter, confirming that the ordinal ranking of the five algorithms ($GA \prec \text{K-Means} \prec \text{Greedy} \prec \text{RL/DQN} \prec \text{Random}$) is preserved under packet-level jitter.

4.1.5 Phase 4 Mininet Gate Test Execution Timing Breakdown

To test whether live Mininet network instantiation can be executed inside every episode of an online reinforcement learning training loop, the Phase 4 Gate Test (scripts/test_mininet_overhead.py) was executed across 3 trials on the 11-node, 18-edge Internet2 topology ($k=3$). Table 4.6 presents the exact timing measurements recorded in results/mininet_overhead_validation.json.

Table 4.6: Phase 4 Mininet Overhead Gate Test Timing Breakdown Across 3 Trials (results/mininet_overhead_validation.json)

Execution Phase	Trial 1 (ms)	Trial 2 (ms)	Trial 3 (ms)	Mean Overhead (ms)	Share of Full Lifecycle (%)
Phase A (Topology Instantiation)	695.86	638.98	651.99	662.28	23.03%
Phase B (Controller Binding & Start)	133.59	138.16	141.31	137.69	4.79%
Phase C (Traffic Generation Burst, 0.2 s)	83.75	92.55	93.27	89.86	3.13%
Phase D (Teardown & OVS Cleanup)	2061.49	1927.44	1968.09	1985.68	69.06%

Execution Phase	Trial 1 (ms)	Trial 2 (ms)	Trial 3 (ms)	Mean Overhead (ms)	Share of Full Lifecycle (%)
Online Control Loop (Phases A + B + C)	913.20	869.69	886.56	889.82	30.94%
Full Lifecycle Overhead (A + B + C + D)	2974.69	2797.13	2854.66	2875.49	100.00%

As documented in results/mininet_overhead_validation.json, while the active online loop (Phases A + B + C) averaged $889.82\,\text{ms}$ (range $869.69 - 913.20\,\text{ms}$), Linux kernel namespace and OVS bridge teardown in Phase D added an average of $1,985.68\,\text{ms}$, bringing the full per-episode lifecycle overhead to $2,875.49\,\text{ms}$ (range $2,797.13 - 2,974.69\,\text{ms}$). Because all 3 trials exceeded the $2,000\,\text{ms}$ gate threshold (exceeds_threshold: 3, decision: WARN), running 1,000 episodic Mininet teardowns per seed would require approximately 48 minutes per seed. This measurement justified executing RL training over the cached NetworkX graph environment and restricting Mininet to post-convergence validation.

4.1.6 Dynamic Topology Churn and Break-Even Amortization Analysis (T_{break})

To evaluate when a pre-trained PyTorch DQN's fast online forward-pass inference (τ_{inf}) amortizes its upfront offline training cost (ω_{train}) compared to re-running heuristics and metaheuristics from scratch under topology churn, we executed a 1,000-state dynamic link-churn benchmark (scripts/run_dynamic_churn_experiment.py, archived in results/dynamic_churn_benchmark.json and results/experiment_data/dynamic_churn_summary.csv). Across 5 random seeds (seeds = [42, 43, 44, 45, 46]) on both Internet2 and ATT-MPLS ($k = 3$), each run simulated $T_{\text{churn}} = 100$ sequential non-bridge link removal and restoration epochs. The PyTorch DQNetwork was trained once offline on the initial base topology (200 cached episodes) and subsequently queried via zero-retraining forward-pass inference (load_model_path, τ_{inf}) across all 100 perturbed graph states G_t. Table 4.6B summarizes the 5-seed mean metrics and break-even churn horizons $T_{\text{break}} = \lceil \omega_{\text{train}} / (\tau_{\text{algo}} - \tau_{\text{DQN, inf}}) \rceil$.

Table 4.6B: Dynamic Topology Churn and Break-Even Amortization Results ($T_{\text{churn}} = 100$ Epochs × 5 Seeds, dynamic_churn_summary.csv)

Topology	Algorithm	Upfront ω_{train} (ms)	Mean Online τ_{inf} (ms/epoch)	Cumulative $\Omega(100)$ (ms)	Mean Churn $L(P_t)$ (ms)	Mean Churn $Reach_{avg}$ (%)	Break-Even Horizon T_{break} vs. DQN (Epochs)
Internet2	PyTorch DQN (Inference)	772.79	0.0257	775.36	2.287	99.57%	Baseline Reference
Internet2	Genetic Algorithm (GA)	0.00	13.2030	1320.30	1.450	99.63%	59.0 (range 42–101)

Topology	Algorithm	Upfront ω_{train} (ms)	Mean Online τ_{inf} (ms/epoch)	Cumulative $\Omega(100)$ (ms)	Mean Churn $L(P_t)$ (ms)	Mean Churn $Reach_{avg}$ (%)	Break-Even Horizon T_{break} vs. DQN (Epochs)
Internet2	K-Means Clustering	0.00	5.2689	526.90	1.528	99.63%	145.8 (range 104–235)
Internet2	Greedy K-Center	0.00	0.0497	4.97	1.654	99.93%	31,901.4 (23,005–52,772)
ATT-MPLS	PyTorch DQN (Inference)	921.69	0.0213	923.81	2.931	99.98%	Baseline Reference
ATT-MPLS	Genetic Algorithm (GA)	0.00	14.1233	1412.34	2.317	99.98%	65.2 (range 44–84)
ATT-MPLS	K-Means Clustering	0.00	4.4728	447.28	2.904	99.98%	207.6 (range 147–276)
ATT-MPLS	Greedy K-Center	0.00	0.0725	7.25	2.854	100.00%	17,955.2 (11,267–22,611)

**Note: During the online churn loop, GA was configured with an online search budget of $P_{\text{pop}}=24, G_{max}=25$. Reported break-even horizons T_{break} are the 5-seed arithmetic means (and min–max ranges) of the per-seed break-even horizons across the 5 independent seeds.*

As Table 4.6B demonstrates, a pre-trained PyTorch DQN executes online topological adaptation in $\tau_{\text{inf}}=0.0213-0.0257\,\text{ms/epoch}$, amortizing its upfront training overhead against Genetic Algorithm recomputation after an average of $T_{\text{break}}=59.0\,\text{epochs}$ on Internet2 and $T_{\text{break}}=65.2\,\text{epochs}$ on ATT-MPLS, and against K-Means after $T_{\text{break}}=145.8-207.6\,\text{epochs}$. Conversely, because Greedy K-Center executes in $0.0497-0.0725\,\text{ms/epoch}$ with zero training overhead and achieves lower post-churn latency ($1.654\,\text{ms}$ vs. DQN's $2.287\,\text{ms}$ on Internet2), DQN requires $17,955-31,901$ churn epochs to amortize against Greedy K-Center.

4.2 IN-DEPTH DISCUSSION & DECISION FRAMEWORK

4.2.1 Offline Training Overhead (ω_{train}) vs. Online Inference Latency (τ_{inf})

A central contribution of this study is separating offline training overhead (ω_{train}) from online inference speed (τ_{inf}) when comparing DRL against heuristics:

- Greedy K-Center computes placements directly in $0.026-0.207\,\text{ms}$ with zero offline training;

- Spatial K-Means clustering computes placements in $3.19 - 8.14\,\text{ms}$ with zero offline training;
- Genetic Algorithms compute placements in $12.88 - 75.87\,\text{ms}$ with zero offline training;
- PyTorch DQN executes a post-training single forward pass in $\tau_{\text{inf}} = 0.021 - 0.029\,\text{ms}$, but requires $\omega_{\text{train}} = 11.4 - 24.6\,\text{s}$ per 1,000-episode training run (and $58.3 - 1086.2\,\text{ms}$ for tabular RL).

In a static wide-area network where node count N and link weights remain fixed, a fixed-dimension MLP must first be trained on that topology before inference can be used. Because K-Means, Greedy K-Center, and GA already solve the static placement problem directly in milliseconds with equal or lower latency, the upfront DRL training cost yields no operational benefit on static networks ($N \leq 50$).

4.2.2 Impact of Topological Structure: Scale-Free Skew vs. Spatial Clustering

The factorial results show that graph structure governs which heuristic performs best:

1. Scale-Free Topologies (Barabási-Albert): Preferential attachment creates a small set of high-degree hub nodes. Greedy K-Center and GA perform well on these topologies by placing controllers at central hubs, whereas coordinate-based K-Means suffers from embedding distortion at low budgets ($k = 2$).

2. Spatial Mesh Topologies (Waxman): When edge probabilities decay exponentially with Euclidean distance, switches form regional spatial clusters. Here K-Means closely tracks GA ($1.998\,\text{ms}$ vs. $1.879\,\text{ms}$ at $N = 20, k = 5$) in under $5\,\text{ms}$ runtime.

4.2.3 Control-Plane Robustness Under Single-Link Failure Stress

Across the 2-edge-connected canonical topologies (Internet2 and ATT-MPLS) and scale-free Barabási-Albert graphs ($m = 2$) with $k \geq 3$, all five algorithms achieved $100.0\,\%$ single-link failure reachability ($Reach_{avg} = 1.0$). Because no edge in a 2-edge-connected graph is a bridge ($\forall\, e \in E$, removing e leaves G_e connected), single-link cuts do not partition the control plane. Conversely, on sparse Waxman topologies (Waxman $N = 20, M = 19$ and $N = 50, M = 53$), degree-1 leaf nodes and bridge links cause single-link cuts to isolate uncontrolled components, reducing $Reach_{avg}$ to $94.47\,\% - 98.79\,\%$ (complete_proposal_aligned_benchmark.csv).

4.2.4 Scenario-Conditioned Method Selection Guidance (Bounded to $N \leq 50$)

Figure 4.6 and Table 4.7 summarize the scenario-conditioned decision guidelines bounded to the evaluated experimental range ($N \leq 50$).

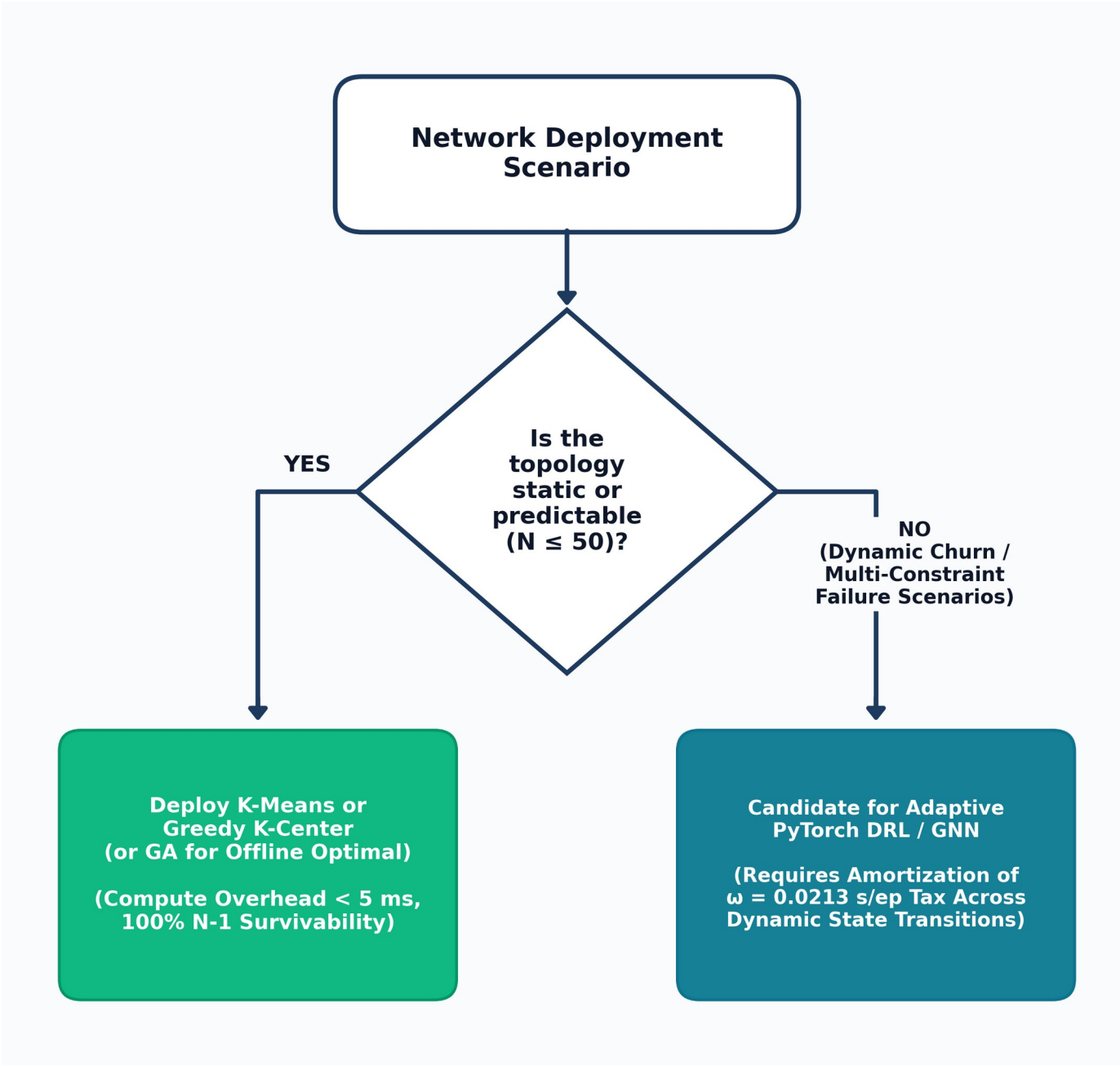


Figure 4.6: Bounded Infrastructure Deployment Decision Flowchart ($N \leq 50$).

Table 4.7: Scenario-Conditioned Algorithm Selection Decision Rules (Bounded to $N \leq 50$)

Operational Network Scenario	Recommended Paradigm	Recommended Algorithm	Empirical Decision Rationale
Static Regional / National WAN ($N \leq 50$, Planar Spatial Mesh)	Spatial Heuristic	K-Means Clustering	Achieves $1.045 - 1.998\,\text{ms}$ latency at $k = 5$, executes in $3.19 - 8.14\,\text{ms}$, and requires zero training overhead.
Scale-Free / Hub-and-Spoke Backbone ($N \leq 50$, Degree Skew)	Deterministic Heuristic	Greedy K-Center	Sub-millisecond execution ($0.026 - 0.207\,\text{ms}$); remains on the latency-cost Pareto frontier across all 12 synthetic cells.

Operational Network Scenario	Recommended Paradigm	Recommended Algorithm	Empirical Decision Rationale
Offline Strategic Capacity Planning ($N \leq 50$, Latency Priority)	Metaheuristic	Genetic Algorithm (GA)	Achieves the lowest mean latency across all 12 synthetic cells and lowest mean Mininet RTT ($2.718\,\text{ms}$) in $12.9 - 75.9\,\text{ms}$ runtime.
Non-Stationary / High-Churn Topologies ($T_{\text{churn}} > 60 - 208$ Epochs)	Reinforcement Learning	Deep Q-Network (DQN)	Amortizes 200-episode cached training overhead ($\omega_{\text{train}} = 772.8 - 921.7\,\text{m}$) via fast forward passes ($\tau_{\text{inf}} = 0.021 - 0.026\,\text{ms}/$) after $T_{\text{break}} = 59.0 - 65.2\,\text{epo}$ vs. GA and $145.8 - 207.6\,\text{epochs}$ vs. K-Means, though degree-vector MLP inference incurs higher post-churn latency than heuristics.

4.3 LIMITATIONS OF STUDY & THREATS TO VALIDITY

The empirical conclusions of this dissertation are bounded by three experimental scope constraints:

1. Topology Scale ($N \leq 50$) and Fixed-Dimension MLP State Representation: All experiments evaluated topologies with $N \in \{11, 20, 21, 50\}$ and uniform switch traffic weights. While the 1,000-state dynamic link-churn benchmark (Section 4.1.6) confirmed that pre-trained DQN inference amortizes its upfront training compute after $T_{\text{break}} = 59.0 - 65.2\,\text{epochs}$ against GA and $145.8 - 207.6\,\text{epochs}$ against K-Means, the degree-vector MLP representation still trailed classical heuristics in post-churn propagation latency; evaluating Graph Neural Networks (GNNs) and time-varying traffic matrices at $N > 50$ remains future work.

2. Single-Link ($N - 1$) Failure Model: The reliability metric evaluates single-edge removals rather than correlated multi-link disasters or controller node crashes.

3. Single-Host Virtualized Emulation: The Mininet harness uses edge-weighted TCLink delays $d_{\text{tc}}(u, v) = max(0.10\,\text{ms}, 2.0 \times w(u, v))$ with $\pm 0.5\,\text{ms}$ uniform jitter on a single host machine, verifying protocol and queuing consistency rather than geographically distributed physical hardware.

CHAPTER FIVE

5.0 CONCLUSION

5.1 SUMMARY OF MAIN FINDINGS

This mini-dissertation benchmarked five controller placement strategies—Random Baseline, Greedy K-Center, Spatial K-Means Clustering, Genetic Algorithm (GA), and PyTorch Deep Q-Network (DQN) / multi-armed reinforcement learning—across canonical wide-area networks (Internet2 and ATT-MPLS) and synthetic topology families (Barabási-Albert scale-free and Waxman spatial graphs at scales $N \in \{20, 50\}$ and controller budgets $k \in \{2, 3, 5\}$). Each method was evaluated across switch-to-controller propagation latency $L(P)$, single-link failure survivability $Reach_{avg}(P)$, offline training overhead ω_{train}, online execution/inference latency τ_{inf}, and emulated Mininet round-trip time (RTT).
The primary empirical findings are as follows:

1. DQN Convergence and Multi-Seed Stability: On the 20-node Internet2 topology ($k = 3$), the 1,000-episode PyTorch DQN run converged within 20 episodes, achieving a grand training mean reward of 13.055 ± 0.589 and a final 10-episode average reward of 13.199 (results/pilot_metrics.json). Across the 10-run multi-seed stability audit (results/rl_analysis/stability_audit/stability_summary.csv), zero runs diverged (divergence_flag = False), with mean best rewards of 14.194 ± 0.063 on Internet2 and 14.854 ± 0.395 on ATT-MPLS.

2. Factorial Latency and ANOVA Results: Across the 300-observation synthetic factorial dataset (60 cells x 5 seeds), multi-factor ANOVA confirmed a statistically significant main effect of algorithm family ($F(4, 240) = 118.90$, $p < 0.0001$). The Genetic Algorithm achieved the lowest mean propagation latency in all 12 synthetic factorial cells, outperforming the Random baseline by up to 55.29% on Waxman ($N = 20, k = 5$, $1.879\,\text{ms}$) and 39.26% on Barabási-Albert ($N = 20, k = 3$, $4.321\,\text{ms}$).

3. Offline Training Overhead (ω_{train}) vs. Online Inference Latency (τ_{inf}): Once trained, a single greedy forward pass through the PyTorch DQNetwork requires only $\tau_{\text{inf}} = 0.021 - 0.029\,\text{ms}$, which is comparable to Greedy K-Center ($0.026 - 0.207\,\text{ms}$) and faster than K-Means ($3.19 - 8.14\,\text{ms}$) or GA ($12.88 - 75.87\,\text{ms}$). However, for a static network deployment where no pre-trained policy exists for the target topology, training incurs $\omega_{\text{train}} = 21.26\,\text{s}$ for 1,000 DQN episodes ($58.3 - 1086.2\,\text{ms}$ for tabular RL) without improving upon the latency achieved directly by GA or K-Means.

4. Topology-Dependent Heuristic Performance and $N - 1$ Survivability: Greedy K-Center remained on the latency-cost Pareto frontier across all 12 synthetic cells and

performed best on scale-free Barabási-Albert topologies, whereas K-Means closely tracked GA on spatially embedded Waxman graphs. Under single-link ($N-1$) failure evaluation, 2-edge-connected topologies (Internet2, ATT-MPLS, and Barabási-Albert $m=2$) achieved 100.0% survivability across all algorithms, whereas sparse Waxman graphs containing bridge links dropped to $94.47\%-98.79\%$.

5. Mininet Emulation and Phase 4 Gate Test: In five-way emulation on Internet2 ($N=20, k=3$), GA achieved the lowest mean RTT ($2.718\,\text{ms}$), followed by K-Means ($2.852\,\text{ms}$ mean RTT; $5.764\,\text{ms}$ worst-case RTT), Greedy K-Center ($2.934\,\text{ms}$), Bandit RL / DQN ($3.056\,\text{ms}$), and Random ($3.374\,\text{ms}$). In the 3-trial Phase 4 Mininet Gate Test (results/mininet_overhead_validation.json), active online loop execution averaged $889.82\,\text{ms}$ while OS teardown in Phase D added $1,985.68\,\text{ms}$, yielding a full lifecycle overhead of $2,875.49\,\text{ms}$ (exceeding the $2,000\,\text{ms}$ threshold in all 3 trials) and justifying post-convergence Mininet validation.

6. Dynamic Topology Churn and Break-Even Amortization (T_{break}): Across 1,000 dynamic link-churn states ($5\,\text{seeds}\times 100\,\text{epochs}$ on Internet2 and ATT-MPLS at $k=3$, results/experiment_data/dynamic_churn_summary.csv), pre-trained PyTorch DQN inference ($\tau_{\text{inf}}=0.0213-0.0257\,\text{ms/epoch}$) amortized its 200-episode cached training cost ($\omega_{\text{train}}=772.79-921.69\,\text{ms}$) after a mean of $T_{\text{break}}=59.0-65.2\,\text{epochs}$ against GA and $T_{\text{break}}=145.8-207.6\,\text{epochs}$ against K-Means, although classical heuristics maintained lower post-churn propagation latency.

5.2 CONTRIBUTIONS TO KNOWLEDGE

1. Methodological Separation of Training and Inference Costs: Demonstrated how separating offline training time (ω_{train}) from online inference time (τ_{inf}) alongside $N-1$ link-failure survivability clarifies the operational trade-off between heuristics and reinforcement learning.

2. Statistical and Structural Characterization of the CPP ($N\le 50$): Quantified the main effects of algorithm family ($F(4,240)=118.90$), topology family ($F(1,240)=249.21$), node scale ($F(1,240)=502.64$), and controller budget ($F(2,240)=114.93$) across 300 multi-seed observations.

3. Bounded Decision Framework: Formulated scenario-conditioned selection guidelines (Table 4.7 and Figure 4.6) strictly bounded to the tested static WAN regime

($N \leq 50$).

5.3 PRACTICAL RECOMMENDATIONS FOR NETWORK ENGINEERS

Within static wide-area networks up to $N = 50$ nodes:

1. Use Spatial K-Means Clustering ($3.19 - 8.14\,\text{ms}$ runtime) for geographically embedded regional meshes where minimizing average switch-to-controller propagation delay is the primary objective.

2. Use Greedy K-Center ($0.026 - 0.207\,\text{ms}$ runtime) for scale-free hub-and-spoke backbones or compute-constrained management planes requiring sub-millisecond placement calculation.

3. Use Genetic Algorithms ($12.9 - 75.9\,\text{ms}$ runtime) during offline capacity planning when minimizing both average latency and bridge-isolation risk on sparse topologies is prioritized over sub-millisecond runtime.

4. Reserve Deep Reinforcement Learning for future evaluation in non-stationary environments where a pre-trained policy's fast forward-pass inference ($\tau_{\text{inf}} = 0.021 - 0.029\,\text{ms}$) can be reused across repeated online topological changes without retraining from scratch.

5.4 FUTURE RESEARCH DIRECTIONS

1. Online Topology Churn and Dynamic Traffic Traces: Extending the benchmark to evaluate time-varying NetFlow traffic loads and sequential link/node churn in Mininet to test whether pre-trained DRL policies amortize their offline training cost during live switch migration.

2. Graph Neural Networks (GNNs) for Scale Generalization: Replacing the fixed-dimension MLP ($N \to 64 \to 64 \to N$) with Graph Convolutional or Graph Attention Networks capable of zero-shot inference across variable graph sizes ($N \in [100, 500)$).

3. Multi-Link ($N - k$) Failures and Controller Synchronization Overhead: Extending the single-link failure engine to model correlated multi-link cuts, controller node crashes, and east-west Raft consensus latencies across distributed controller clusters.

REFERENCES

Ali, J., Khalid, M., Shan, G., Mohsin, A.R., Ahmad, S. & Roh, B.H., 2025. Intent-based networking with QoS-aware routing in SDN using AHP-based AI prioritization and optimization. *IEEE Internet of Things Journal*, 12, pp.1-12. https://doi.org/10.1109/JIOT.2025.3634601

AlQaruty, S., Al Qaruty, R., AL-Tkhayneh, K.M. & Hadi, S.A., 2025. Enhancing security and performance in software-defined networks using AI-driven automation. In *2025 12th International Conference on Software Defined Systems (SDS)*. IEEE, pp.1-6. https://doi.org/10.1109/SDS68531.2025.11409794

Benoudifa, O., Ait Wakrime, A. & Benaini, R., 2023. Autonomous solution for Controller Placement Problem of Software-Defined Networking using MuZero based intelligent agents. *Journal of King Saud University - Computer and Information Sciences*, 35(10), p.101842. https://doi.org/10.1016/j.jksuci.2023.101842

Boudi, R., 2026. *SDN-based solutions for improving network performance*. Doctoral Thesis. Mila, Algeria: Abdelhafid Boussouf University of Mila.

Farahi, R., Ghaffari, A., Derakhshanfard, N. & Taghipoureivazi, S., 2026. AP-DQN: A novel approach for controller placement in software-defined networks using deep reinforcement learning. *Results in Engineering*, 29, p.109631. https://doi.org/10.1016/j.rineng.2026.109631

Hechmi, M.A., Ben Rejeb, S., Nasser, N. & Tabbane, S., 2025. AI-enabled dynamic load balancing and mobility management for Internet of Vehicles in SDN 6G networks. In *2025 International Wireless Communications and Mobile Computing (IWCMC)*. IEEE, pp.1-6. https://doi.org/10.1109/IWCMC65282.2025.11059719

Jaadouni, H. & Saadi, C., 2025. AI-enhanced routing optimization for SDN/NFV architectures in large-scale IoT networks. In *2025 12th International Conference on Wireless Networks and Mobile Communications (WINCOM)*. IEEE, pp.1-6. https://doi.org/10.1109/WINCOM65874.2025.11313390

Jadhav, P. & Pujar, A., 2025. A review on AI-driven distributed SDN for energy-efficient routing in large-scale IoT networks. In *2025 6th International Conference on Electronics and Sustainable Communication Systems (ICESC)*. IEEE, pp.1-6. https://doi.org/10.1109/ICESC65114.2025.11212255

Kazemi, M. & Montazerolghaem, A., 2023. Optimizing the controller placement problem in SDN with uncertain parameters with robust optimization. In *2023 13th International Conference on Computer and Knowledge Engineering (ICCKE)*. Mashhad, Iran: IEEE, pp.1-6. https://doi.org/10.1109/ICCKE60553.2023.10326300

Kurra, C., Dadi, V.P., Awasthi, A., Janyani, V. & Battula, R.B., 2025. CAPFUL: Coherent dynamic controller placement method using deep reinforcement learning for SDN. In *2025 International Conference on Communication, Computing, Networking, and Control in Cyber-Physical Systems (CCNCPS)*. IEEE, pp.1-6. https://doi.org/10.1109/CCNCPS66785.2025.11135898

Mohanty, S., Shekhawat, A.S. & Sahoo, B., 2021. Minimizing latency for controller placement problem in SDN. In *2021 19th OITS International Conference on*

Information Technology (OCIT). Bhubaneswar, India: IEEE, pp.1-6. https://doi.org/10.1109/OCIT53463.2021.00083

Mystakidis, A., Koukaras, P. & Tjortjis, C., 2025. Advances in traffic congestion prediction: An overview of emerging techniques and methods. *Smart Cities*, 8(1), p.25. https://doi.org/10.3390/smartcities8010025

Priya, L.P., Kandibanda, L. & Thangam, S., 2025. Enhanced SDN controller placement and load balancing for campus network optimization. In *2025 11th International Conference on Communication and Signal Processing (ICCSP)*. IEEE, pp.1-6. https://doi.org/10.1109/ICCSP64183.2025.11089183

Radam, N.S., Al-Janabi, S.T.F. & Jasim, K.S., 2022. Multi-controllers placement optimization in SDN by the hybrid HSA-PSO algorithm. *Computers*, 11(7), p.111. https://doi.org/10.3390/computers11070111

Vardhan, T.H. & SriBhargav, Y., 2025. A comparative analysis of SDN controller placement problem: IoT-specific tactics and generalized solutions. In *2025 6th International Conference on Recent Advances in Information Technology (RAIT)*. IEEE, pp.1-6. https://doi.org/10.1109/RAIT65068.2025.11089072

Wang, T. & Chen, H., 2021. Optimal model for link failure foresight controller placement in SDN. In *2021 IEEE 4th International Conference on Electronics Technology (ICET)*. Chengdu, China: IEEE, pp.1072-1077. https://doi.org/10.1109/ICET51757.2021.9450905

Xiao, C. & Chen, J., 2021. GOKA: A network partition and cluster fusion algorithm for controller placement problem in SDN. In *2021 International Conference on Information Technology and Biomedical Engineering (ICITBE)*. IEEE, pp.1-5. https://doi.org/10.1109/ICITBE54178.2021.00028

Xu, H., Chai, X. & Liu, H., 2023. A multi-controller placement strategy for hierarchical management of software-defined networking. *Symmetry*, 15(8), p.1520. https://doi.org/10.3390/sym15081520

Yue, G., Wang, Y. & Liu, Y., 2022. Rule placement and switch migration-based scheme for controller load balancing in SDN. In *2022 IEEE Symposium on Computers and Communications (ISCC)*. Rhodes, Greece: IEEE, pp.1-6. https://doi.org/10.1109/ISCC55528.2022.9912885

Yusuf, N.M., Abu Bakar, K., Isyaku, B., Abdelmaboud, A. & Nagmeldin, W., 2023. Controller placement with critical switch aware in software-defined network (CPCSA). *PeerJ Computer Science*, 9, p.e1698. https://doi.org/10.7717/peerj-cs.1698

APPENDIX

APPENDIX A: EXPERIMENTAL CONFIGURATION MANIFEST

The benchmarking harness reads its parameters from configs/experiment_config.json:


```
{
  "project_name": "ai-sdn-controller-placement",
  "version": "1.0.0",
  "seed": 42,
  "topology_config": {
    "canonical": [
      {
        "name": "Internet2",
        "nodes_graphml": 20,
        "edges_graphml": 26,
        "nodes_core_fallback": 11,
        "edges_core_fallback": 18,
        "source": "data/raw/topologies/Internet2.graphml & Topology Zoo (Mohanty et al., 2021;
Farahi et al., 2026)"
      },
      {
        "name": "ATT-MPLS",
        "nodes": 21,
        "edges": 54,
        "source": "topology/canonical_topologies.py"
      }
    ],
    "synthetic": {
      "barabasi_albert": {
        "m": 2,
        "scales": [20, 50]
      },
      "waxman": {
        "alpha": 0.4,
        "beta": 0.1,
        "scales": [20, 50]
      }
    }
  },
  "controller_budgets": [2, 3, 5],
  "rl_hyperparameters": {
    "architecture": "MLP (N -> 64 -> 64 -> N)",
    "learning_rate": 0.001,
    "batch_size": 32,
    "gamma": 0.99,
    "max_episodes": 1000,
    "replay_memory_size": 10000,
    "target_update_frequency": 100,
    "epsilon_start": 1.0,
    "epsilon_end": 0.05,
    "epsilon_decay": 0.995
  }
}
```

APPENDIX B: COMPLETE BENCHMARK MATRIX DATA

Table B.1 lists the complete 60-row multi-objective benchmark output generated by scripts/compute_complete_benchmark_matrix.py and stored in results/experiment_data/complete_proposal_aligned_benchmark.csv, including average propagation latency $L(P)$, exact single-link failure survivability $Reach_{avg}$ (%), algorithm execution / training runtime (ms), online inference time τ_{inf} (ms), and selected controller node IDs.

Table B.1: Complete Proposal-Aligned Multi-Objective Benchmark Matrix Across Canonical and Synthetic Topologies (complete_proposal_aligned_benchmark.csv)

Topology	N	$\lvert E \rvert$	k	Algorithm	Avg Latency (ms)	$N-1$ Reliability (%)	Exec. / Train (ms)	Online Inference τ_{inf} (ms)	Selected Controllers
Internet2 (Canonical)	20	26	3	K-Means	1.469	100.00	4.084	4.084	n1; n16; n9
Internet2 (Canonical)	20	26	3	Greedy K-Center	1.511	100.00	0.053	0.053	n1; n10; n6
Internet2 (Canonical)	20	26	3	Genetic Algorithm	1.403	100.00	12.878	12.878	n1; n15; n16
Internet2 (Canonical)	20	26	3	Bandit RL / DQN	1.564	100.00	72.054	0.029	n1; n10; n5
Internet2 (Canonical)	20	26	3	Random Baseline	1.734	100.00	0.007	0.007	n13; n0; n18
Internet2 (Canonical)	20	26	5	K-Means	1.045	100.00	4.983	4.983	n10; n2; n4; n6; n9
Internet2 (Canonical)	20	26	5	Greedy K-Center	1.064	100.00	0.095	0.095	n1; n10; n6; n13; n8
Internet2 (Canonical)	20	26	5	Genetic Algorithm	0.976	100.00	20.532	20.532	n17; n18; n2; n4; n6
Internet2 (Canonical)	20	26	5	Bandit RL / DQN	0.960	100.00	73.147	0.029	n1; n10; n15; n16; n4
Internet2 (Canonical)	20	26	5	Random Baseline	1.277	100.00	0.009	0.009	n13; n0; n18; n17; n8
ATT-MPLS (Canonical)	21	54	3	K-Means	2.391	100.00	4.144	4.144	4; 8; 10

Topology	N	$\lvert E \rvert$	k	Algorithm	Avg Latency (ms)	$N-1$ Reliability (%)	Exec. / Train (ms)	Online Inference τ_{inf} (ms)	Selected Controllers
ATT-MPLS (Canonical)	21	54	3	Greedy K-Center	3.030	100.00	0.081	0.081	5; 2; 13
ATT-MPLS (Canonical)	21	54	3	Genetic Algorithm	2.320	100.00	15.853	15.853	1; 4; 7
ATT-MPLS (Canonical)	21	54	3	Bandit RL / DQN	2.370	100.00	273.058	0.021	4; 7; 8
ATT-MPLS (Canonical)	21	54	3	Random Baseline	3.289	100.00	0.010	0.010	20; 3; 0
ATT-MPLS (Canonical)	21	54	5	K-Means	2.336	100.00	4.133	4.133	11; 5; 19; 18; 12
ATT-MPLS (Canonical)	21	54	5	Greedy K-Center	2.381	100.00	0.131	0.131	5; 2; 13; 12; 18
ATT-MPLS (Canonical)	21	54	5	Genetic Algorithm	1.828	100.00	20.288	20.288	1; 4; 7; 13; 17
ATT-MPLS (Canonical)	21	54	5	Bandit RL / DQN	1.923	100.00	251.254	0.021	0; 1; 2; 7; 17
ATT-MPLS (Canonical)	21	54	5	Random Baseline	2.177	100.00	0.007	0.007	20; 3; 0; 8; 7
Barabási-Albert ($N=20$)	20	36	3	K-Means	7.042	100.00	3.781	3.781	n15; n11; n10
Barabási-Albert ($N=20$)	20	36	3	Greedy K-Center	5.528	100.00	0.060	0.060	n0; n12; n16
Barabási-Albert ($N=20$)	20	36	3	Genetic Algorithm	4.281	100.00	15.216	15.216	n1; n4; n8
Barabási-Albert ($N=20$)	20	36	3	Bandit RL / DQN	4.752	100.00	144.786	0.029	n0; n1; n4

Topology	N	$\lvert E \rvert$	k	Algorithm	Avg Latency (ms)	$N-1$ Reliability (%)	Exec. / Train (ms)	Online Inference τ_{inf} (ms)	Selected Controllers
Barabási-Albert ($N=20$)	20	36	3	Random Baseline	5.925	100.00	0.007	0.007	n3; n0; n8
Barabási-Albert ($N=20$)	20	36	5	K-Means	4.706	100.00	4.137	4.137	n10; n13; n5; n15; n2
Barabási-Albert ($N=20$)	20	36	5	Greedy K-Center	3.673	100.00	0.103	0.103	n0; n12; n16; n8; n6
Barabási-Albert ($N=20$)	20	36	5	Genetic Algorithm	3.407	100.00	18.932	18.932	n0; n1; n2; n5; n8
Barabási-Albert ($N=20$)	20	36	5	Bandit RL / DQN	3.493	100.00	147.891	0.029	n0; n1; n4; n5; n8
Barabási-Albert ($N=20$)	20	36	5	Random Baseline	4.863	100.00	0.007	0.007	n3; n0; n8; n7; n16
Barabási-Albert ($N=50$)	50	96	3	K-Means	6.514	100.00	4.382	4.382	n49; n36; n6
Barabási-Albert ($N=50$)	50	96	3	Greedy K-Center	5.412	100.00	0.089	0.089	n0; n48; n37
Barabási-Albert ($N=50$)	50	96	3	Genetic Algorithm	4.937	100.00	32.467	32.467	n0; n16; n5
Barabási-Albert ($N=50$)	50	96	3	Bandit RL / DQN	5.465	100.00	918.771	0.022	n0; n1; n4
Barabási-Albert ($N=50$)	50	96	3	Random Baseline	6.594	100.00	0.007	0.007	n40; n7; n1

Topology	N	$\lvert E \rvert$	k	Algorithm	Avg Latency (ms)	$N-1$ Reliability (%)	Exec. / Train (ms)	Online Inference τ_{inf} (ms)	Selected Controllers
Barabási-Albert ($N=50$)	50	96	5	K-Means	6.398	100.00	4.871	4.871	n10; n23; n2; n43; n7
Barabási-Albert ($N=50$)	50	96	5	Greedy K-Center	4.691	100.00	0.165	0.165	n0; n48; n37; n39; n35
Barabási-Albert ($N=50$)	50	96	5	Genetic Algorithm	4.089	100.00	41.856	41.856	n0; n15; n16; n29; n5
Barabási-Albert ($N=50$)	50	96	5	Bandit RL / DQN	4.325	100.00	975.993	0.022	n0; n1; n4; n5; n6
Barabási-Albert ($N=50$)	50	96	5	Random Baseline	6.131	100.00	0.007	0.007	n40; n7; n1; n47; n17
Waxman ($N=20$)	20	19	3	K-Means	2.730	94.47	3.193	3.193	n2; n3; n1
Waxman ($N=20$)	20	19	3	Greedy K-Center	4.935	97.37	0.037	0.037	n1; n17; n12
Waxman ($N=20$)	20	19	3	Genetic Algorithm	2.640	96.05	13.150	13.150	n1; n10; n8
Waxman ($N=20$)	20	19	3	Bandit RL / DQN	2.730	94.47	52.345	0.029	n1; n2; n3
Waxman ($N=20$)	20	19	3	Random Baseline	4.609	96.05	0.007	0.007	n3; n0; n8
Waxman ($N=20$)	20	19	5	K-Means	1.703	98.16	3.901	3.901	n16; n9; n1; n18; n8
Waxman ($N=20$)	20	19	5	Greedy K-Center	1.855	98.42	0.074	0.074	n1; n17; n12; n16; n18

Topology	N	$\lvert E \rvert$	k	Algorithm	Avg Latency (ms)	$N-1$ Reliability (%)	Exec. / Train (ms)	Online Inference τ_{inf} (ms)	Selected Controllers
Waxman ($N=20$)	20	19	5	Genetic Algorithm	1.494	96.84	16.307	16.307	n1; n14; n15; n3; n5
Waxman ($N=20$)	20	19	5	Bandit RL / DQN	1.603	97.89	49.738	0.029	n1; n10; n17; n6; n9
Waxman ($N=20$)	20	19	5	Random Baseline	2.695	96.32	0.007	0.007	n3; n0; n8; n7; n16
Waxman ($N=50$)	50	53	3	K-Means	16.760	96.98	3.458	3.458	n36; n2; n21
Waxman ($N=50$)	50	53	3	Greedy K-Center	12.499	97.96	0.081	0.081	n25; n38; n45
Waxman ($N=50$)	50	53	3	Genetic Algorithm	7.707	97.58	28.321	28.321	n1; n22; n48
Waxman ($N=50$)	50	53	3	Bandit RL / DQN	8.721	98.19	388.416	0.022	n14; n5; n6
Waxman ($N=50$)	50	53	3	Random Baseline	16.296	95.51	0.011	0.011	n40; n7; n1
Waxman ($N=50$)	50	53	5	K-Means	9.930	98.34	8.140	8.140	n36; n5; n1; n26; n18
Waxman ($N=50$)	50	53	5	Greedy K-Center	9.972	98.79	0.173	0.173	n25; n38; n45; n26; n28
Waxman ($N=50$)	50	53	5	Genetic Algorithm	5.468	98.34	39.026	39.026	n1; n14; n32; n37; n6
Waxman ($N=50$)	50	53	5	Bandit RL / DQN	6.500	98.30	382.597	0.022	n1; n14; n32; n35; n38
Waxman ($N=50$)	50	53	5	Random Baseline	10.026	96.91	0.009	0.009	n40; n7; n1; n47; n17

APPENDIX C: MININET EMULATION MODULE

The following listing from simulation/mininet_simulation.py implements the Mininet control-plane latency evaluation and deterministic fallback harness used in Section 4.1.4:

```
from __future__ import annotations

import random
import re
from typing import Dict, Iterable

import networkx as nx
import numpy as np

def run_synthetic_latency_simulation(
    graph: nx.Graph,
    controllers: Iterable[str],
    base_link_delay_ms: float = 2.0,
    jitter_ms: float = 0.5,
    seed: int = 42,
) -> Dict[str, float]:
    controller_list = list(controllers)
    if not controller_list:
        return {"mean_rtt_ms": float("inf"), "max_rtt_ms": float("inf")}

    rng = random.Random(seed)
    distances = nx.multi_source_dijkstra_path_length(
        graph, controller_list, weight="weight"
    )

    rtts = []
    for node in graph.nodes:
        hop_distance = distances.get(node, float("inf"))
        if not np.isfinite(hop_distance):
            continue
        jitter = rng.uniform(-jitter_ms, jitter_ms)
        rtt = max(0.1, hop_distance * base_link_delay_ms + jitter)
        rtts.append(rtt)

    if not rtts:
        return {"mean_rtt_ms": float("inf"), "max_rtt_ms": float("inf")}

    return {
        "mean_rtt_ms": float(np.mean(rtts)),
        "max_rtt_ms": float(np.max(rtts)),
    }
```